

Week 7

Autoencoders & RNNs

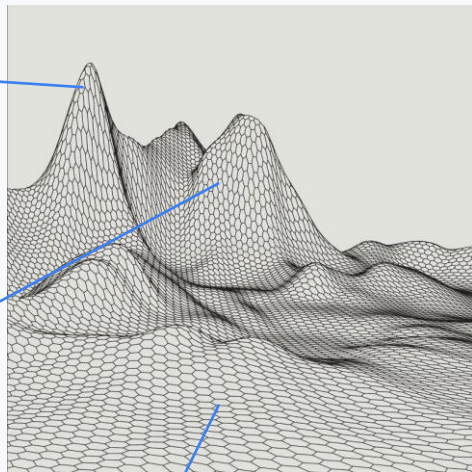
Nebius Academy



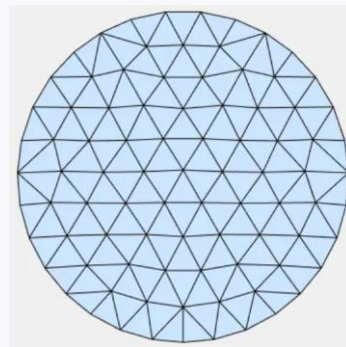
Autoencoders

A rough idea of latent spaces

Strange world of real images



A simple distribution



Latent space



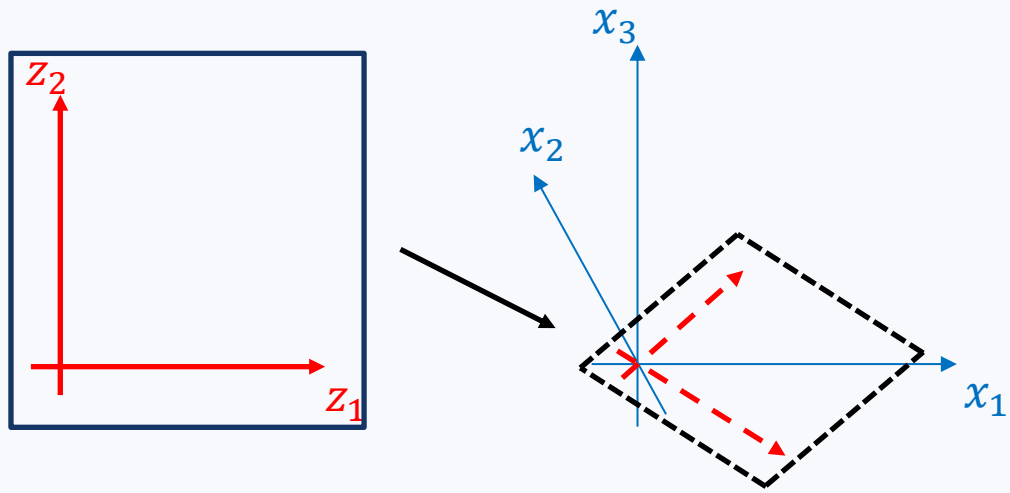
Principal component analysis

Establish

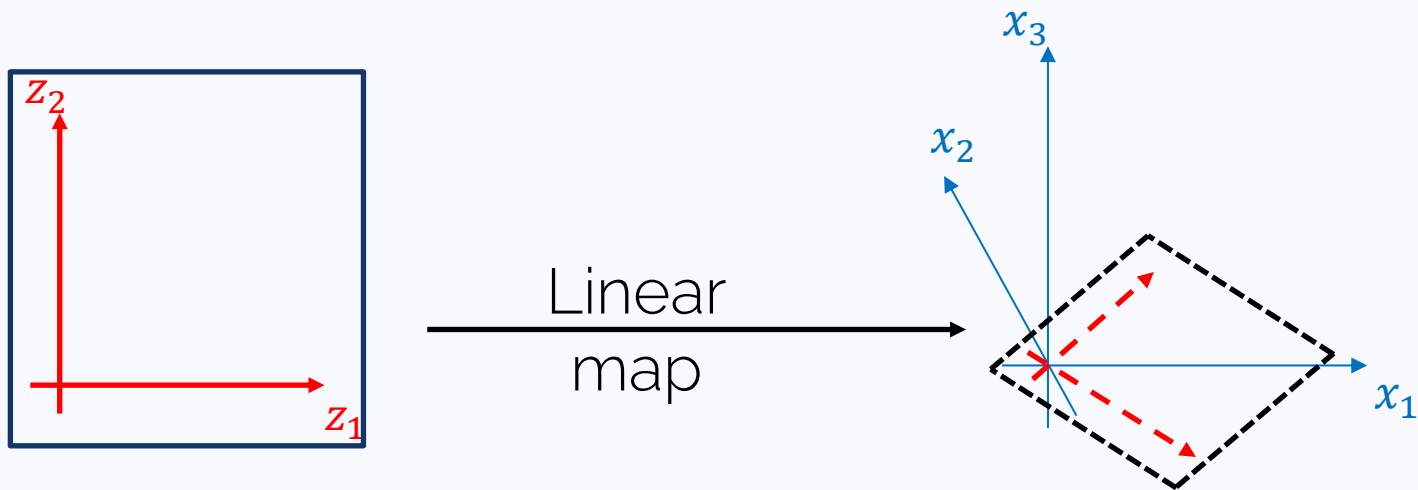
$$X^{N \times D} = Z^{N \times d} \cdot V^T$$

where z_i are new (**latent**) features of the i -th object and $d < D$.

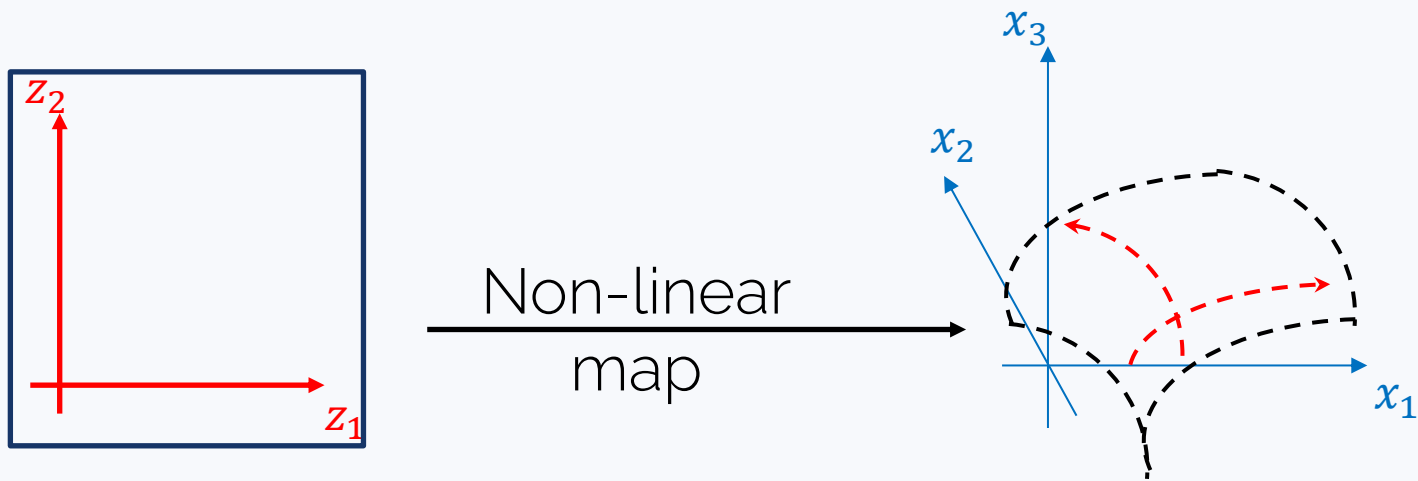
- $Z \rightarrow Z \cdot V^T$ is feature mixing
- $Z \rightarrow U \cdot Z$ is object mixing



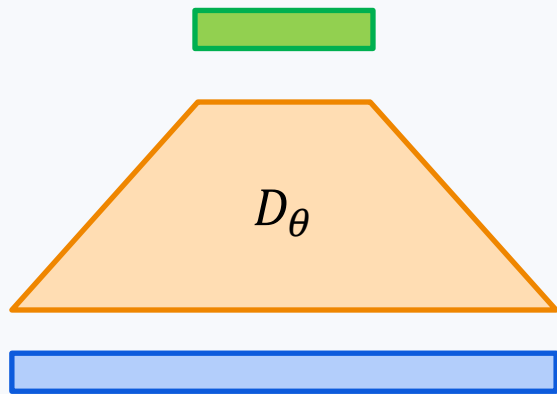
Principal component analysis



General latent variable model



Autoencoder



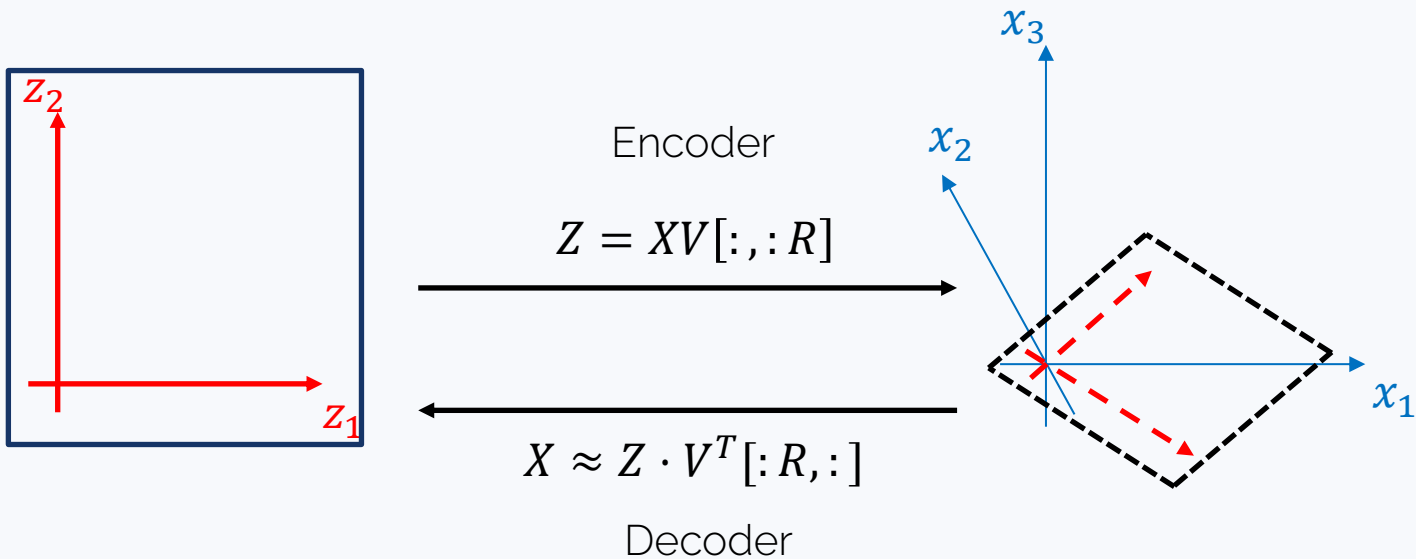
d -dimensional

Decoder

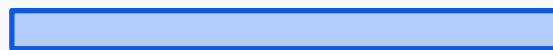
D -dimensional

Principal component analysis

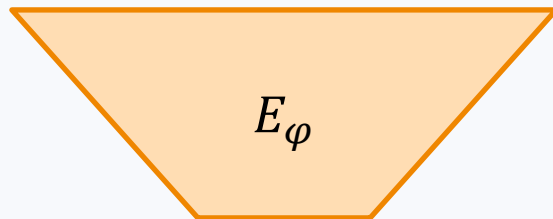
$$X = U\Sigma V^T$$



Autoencoder



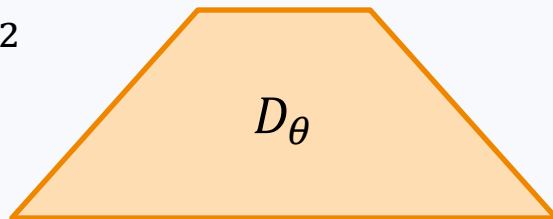
D -dimensional



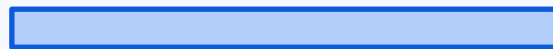
Encoder



d -dimensional



Decoder



D -dimensional

Reconstruction loss:

$$\frac{1}{N} \sum_{i=1}^N \left\| D_\theta \left(E_\varphi(x_i) \right) - x_i \right\|^2$$

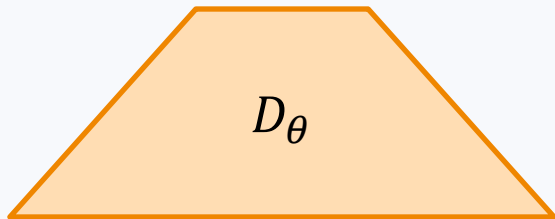
Autoencoder: generation

Some random latent vector

$$z \sim \mathcal{N}(0, I)$$



d -dimensional



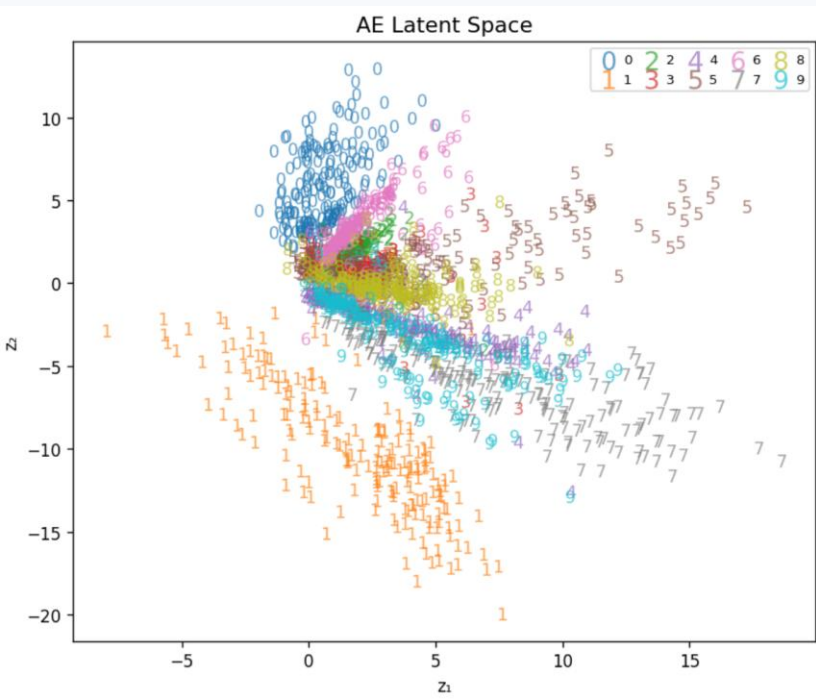
Decoder



D -dimensional

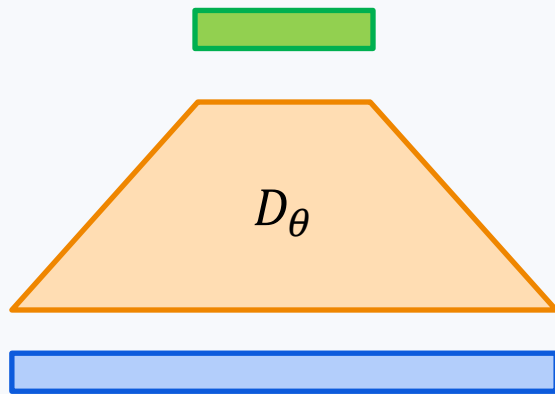
Autoencoder: generation

Doesn't look like $\mathcal{N}(0, I)$



Some random latent vector

$$z \sim \mathcal{N}(0, I)$$

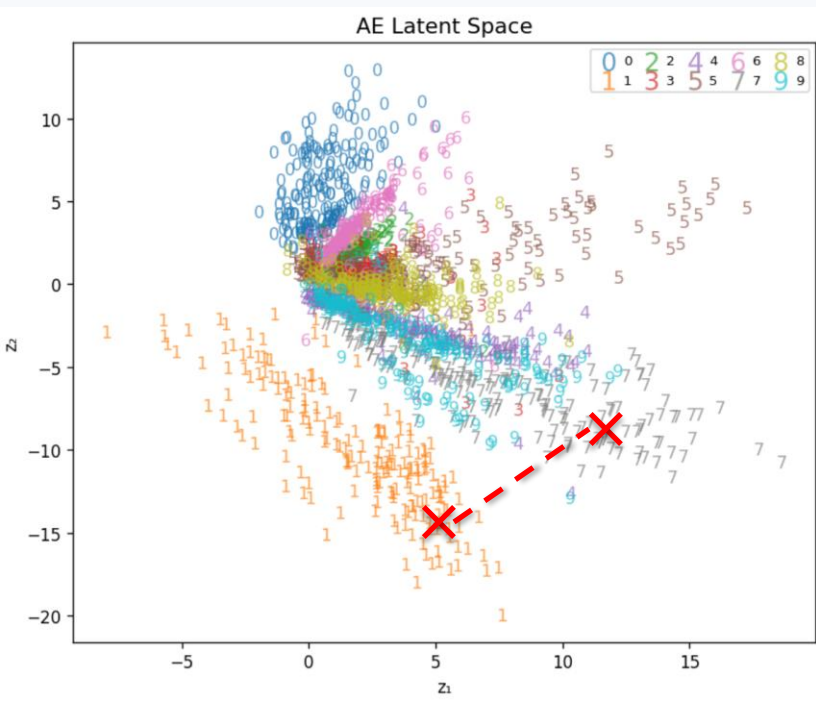


d -dimensional

Decoder

D -dimensional

Autoencoder: latent interpolation

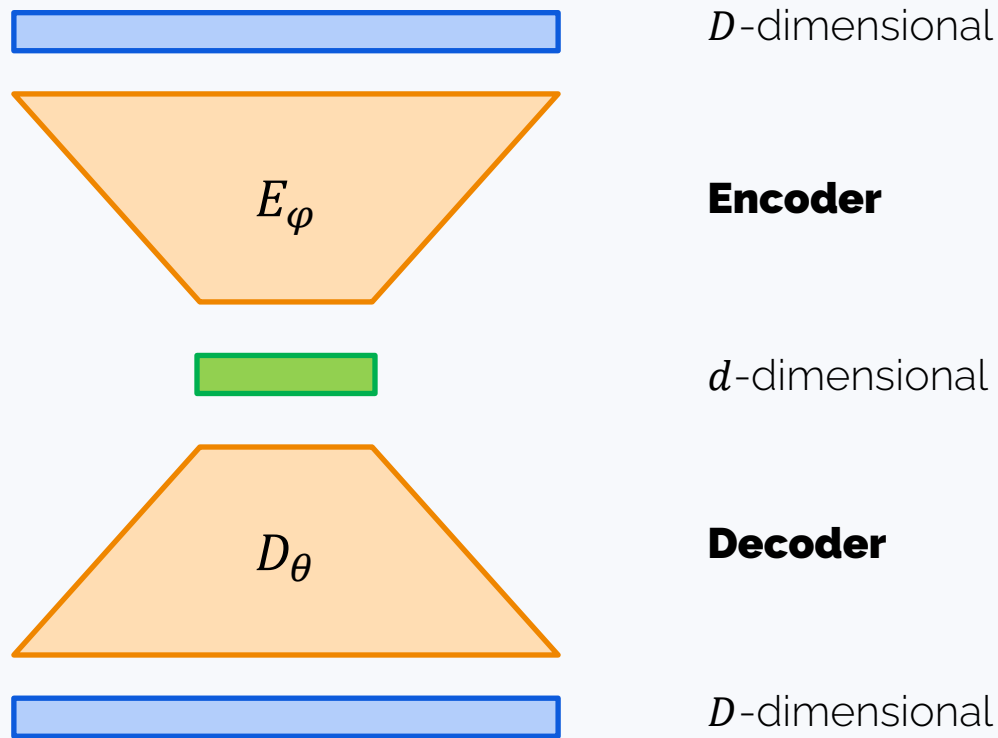


Variational autoencoder

Reconstruction loss:

$$\frac{1}{N} \sum_{i=1}^N \left\| D_{\theta} \left(E_{\varphi}(x_i) \right) - x_i \right\|^2 + \text{regularization term}$$

To ensure that we can sample $z \sim \mathcal{N}(0, I)$

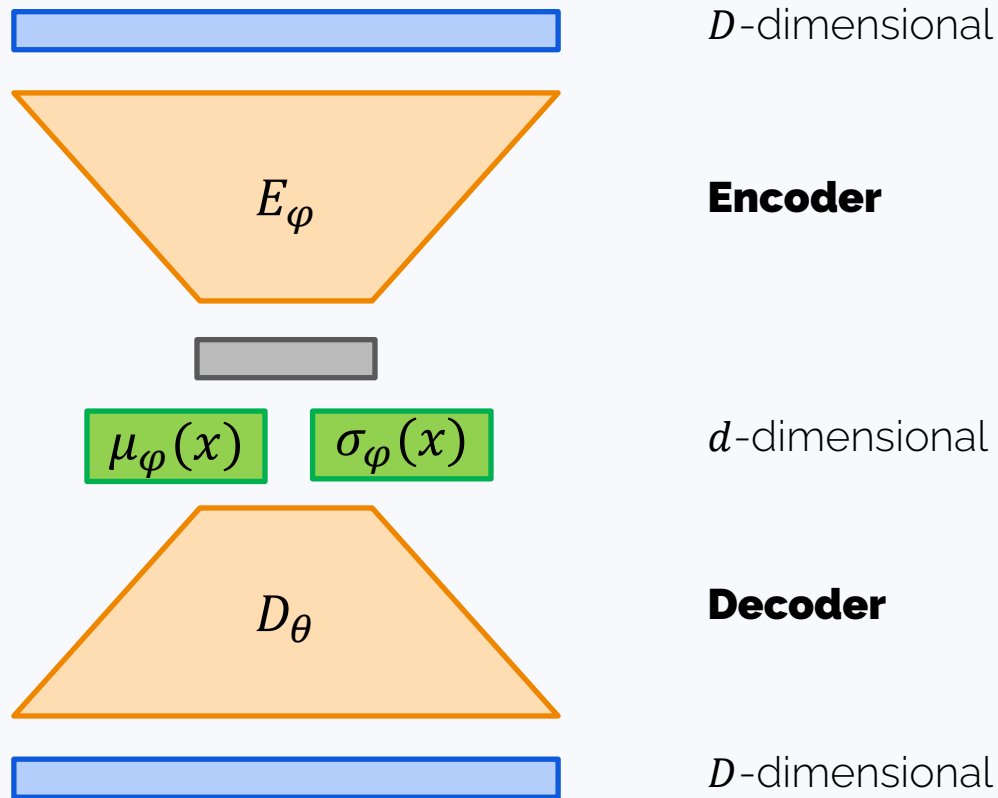


Variational Autoencoder: Forward pass

$$x \rightarrow E_{\varphi}(x)$$

$$z \sim \mathcal{N}(\mu_{\varphi}(x), \sigma_{\varphi}^2(x))$$

$$z \rightarrow D_{\theta}(z) = \hat{x}$$

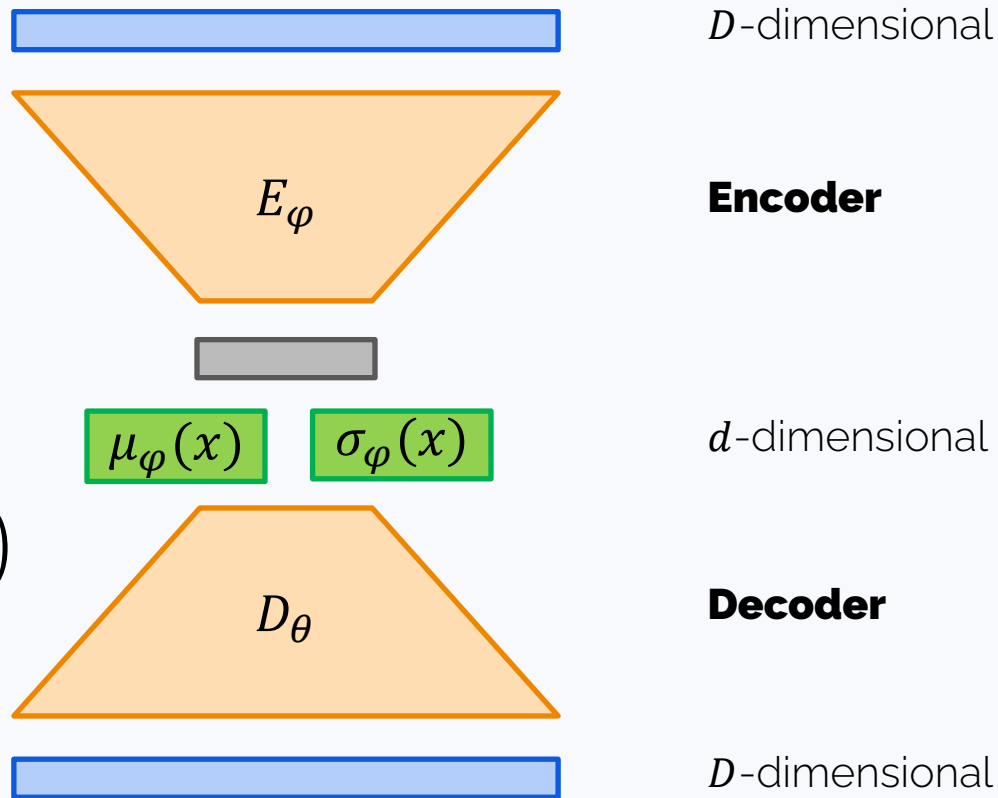


Variational autoencoder

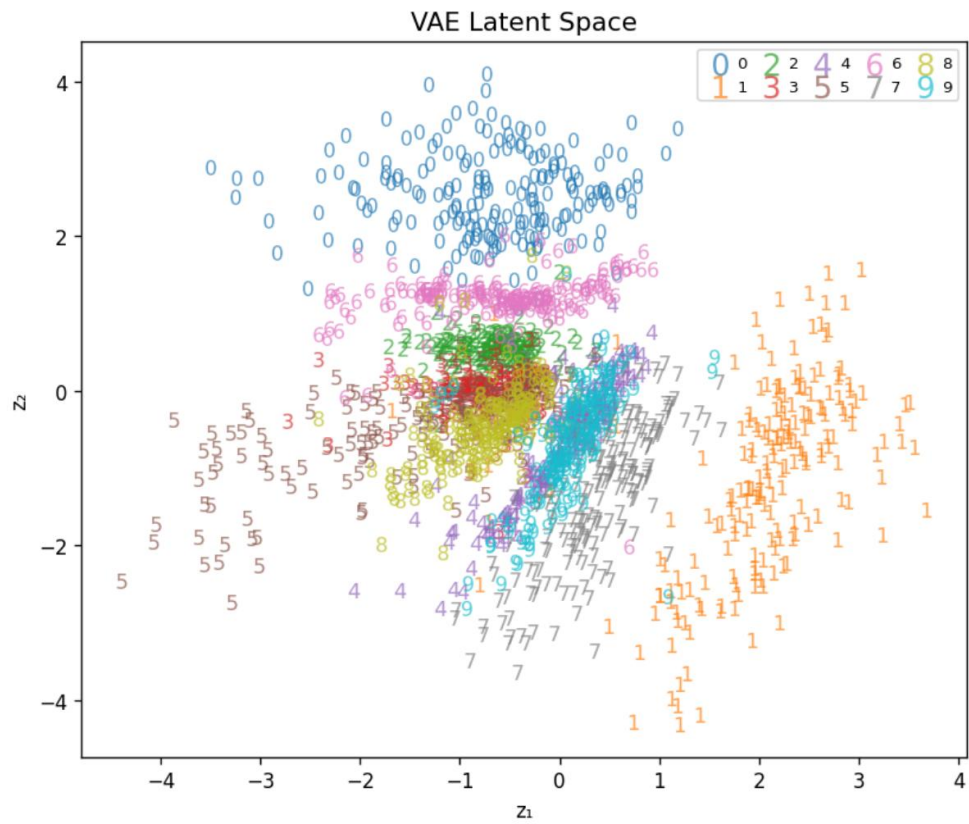
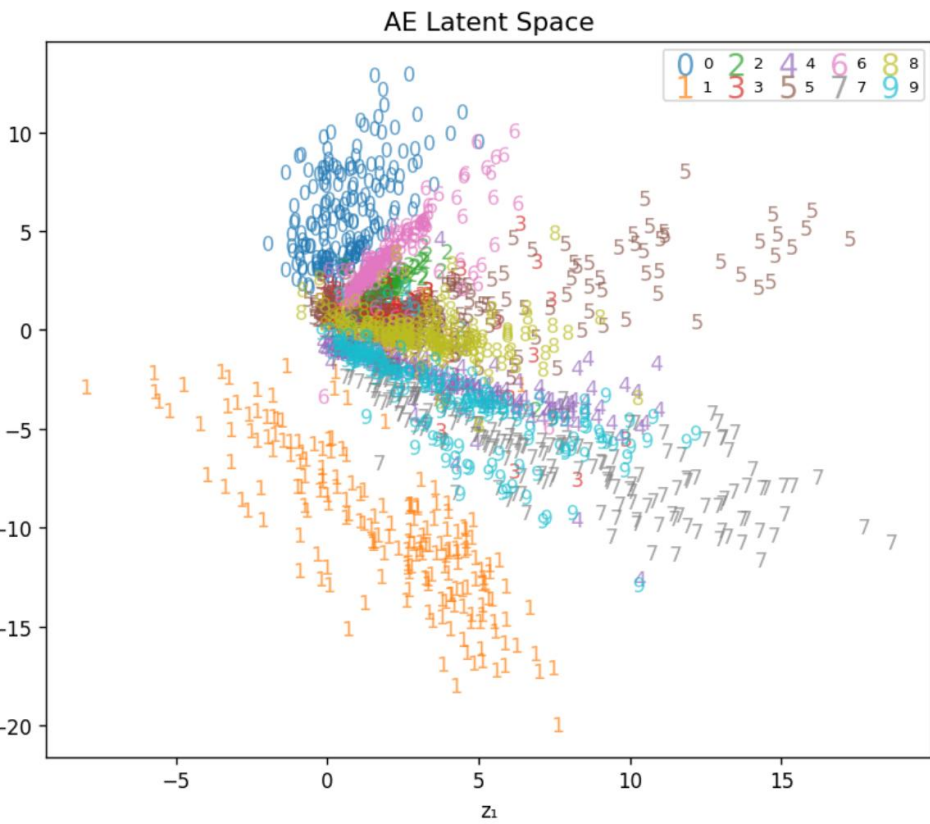
Reconstruction loss:

$$\frac{1}{N} \sum_{i=1}^N \left\| D_{\theta} \left(E_{\varphi}(x_i) \right) - x_i \right\|^2 + \\ + \text{KL} \left(\mathcal{N} \left(\mu_{\varphi}(x), \sigma_{\varphi}(x) \right) \parallel \mathcal{N}(0, I) \right)$$

To ensure that we can sample $z \sim \mathcal{N}(0, I)$



AE vs VAE: latent space



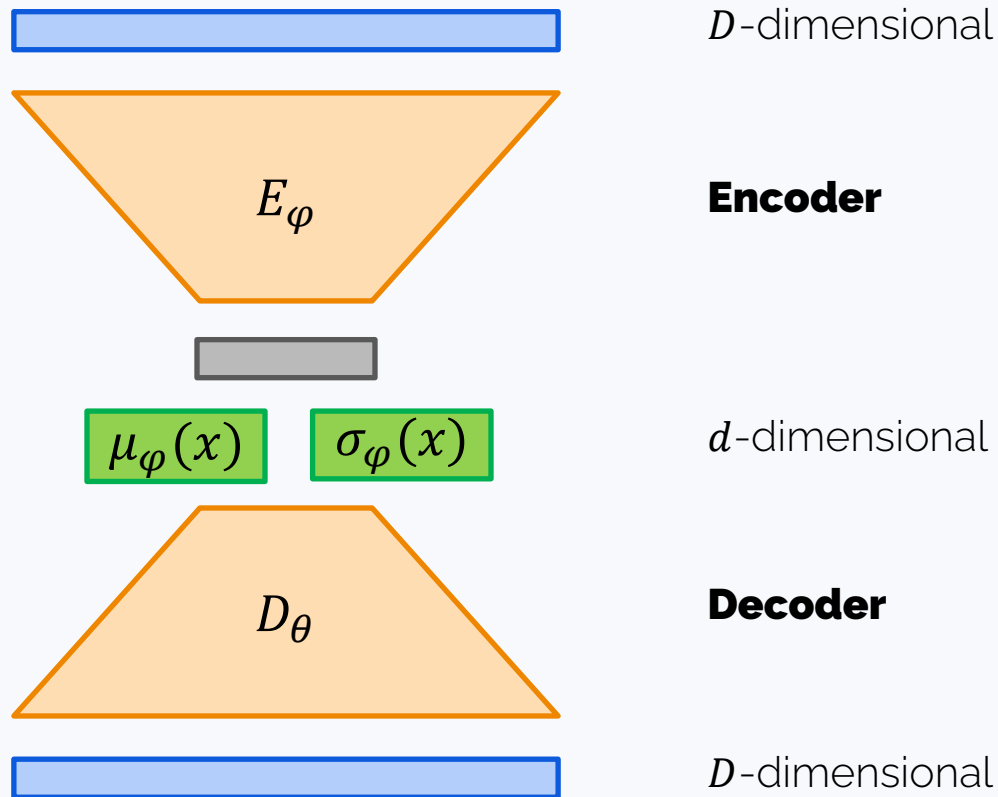
Variational Autoencoder: Forward pass

$$x \rightarrow E_{\varphi}(x)$$

$$z \sim \mathcal{N}(\mu_{\varphi}(x), \sigma_{\varphi}^2(x))$$

$$z \rightarrow D_{\theta}(z) = \hat{x}$$

Gradients won't flow through this!



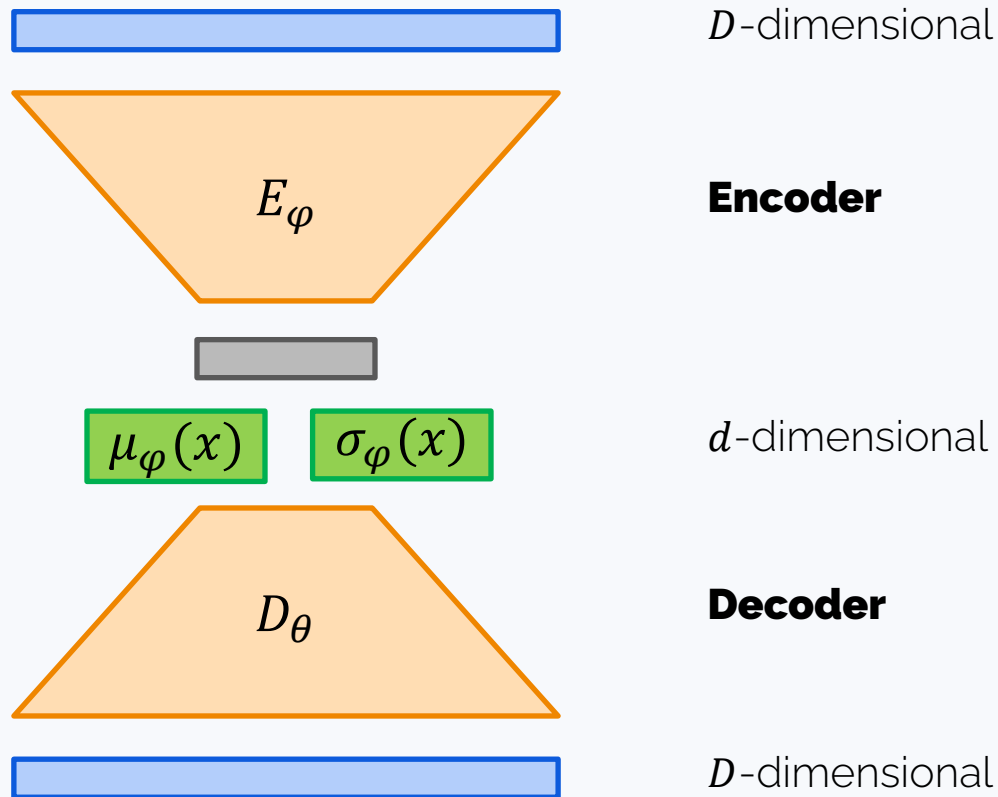
Backward pass

Gradients won't flow through sampling:

$$z \sim \mathcal{N}(\mu_\varphi(x), \sigma_\varphi^2(x))$$

Reparametrization trick:

$$z = \mu_\varphi(x) + \epsilon \cdot \sigma_\varphi(x), \\ \epsilon \sim \mathcal{N}(0; I)$$



Loss for VAE

Importance sampling

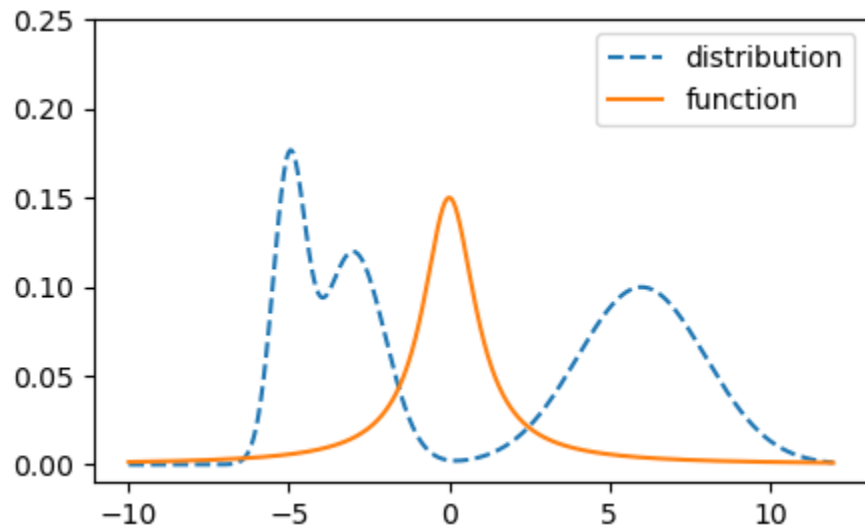
Monte Carlo sampling

We want to calculate

$$\mathbb{E}_{z \sim p(z)} f(z) = \int f(z) p(z) dz$$

Naïve way:

$$\int f(z) p(z) dz \sim \frac{1}{N} \sum_{i=1}^N f(z_i) p(z_i)$$



Too noisy, not reliable!

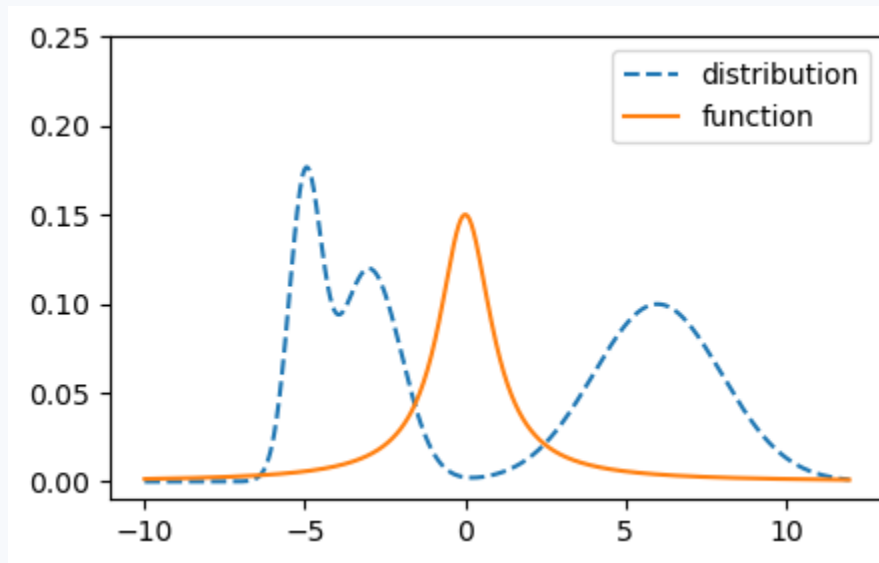
The problem

We want to calculate

$$\mathbb{E}_{z \sim p(z)} f(z) = \int f(z) p(z) dz$$

Monte-Carlo:

$$\int f(z) p(z) dz \sim \frac{1}{N} \sum_{i=1}^N f(z_i), \quad z_i \sim p(z)$$



Still, a problem:

Larger values of $f(z)$ may occur in areas with low density

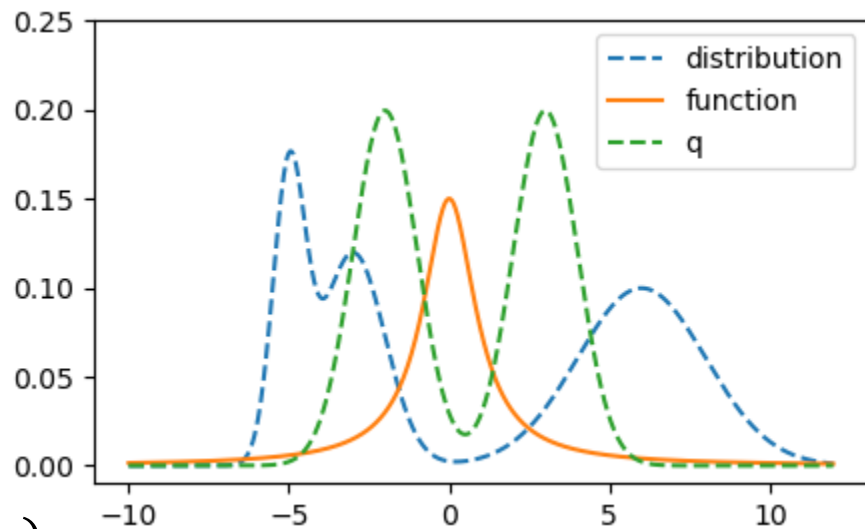
It makes estimation noisy

Importance sampling

Take an auxiliary distribution $q(z)$:

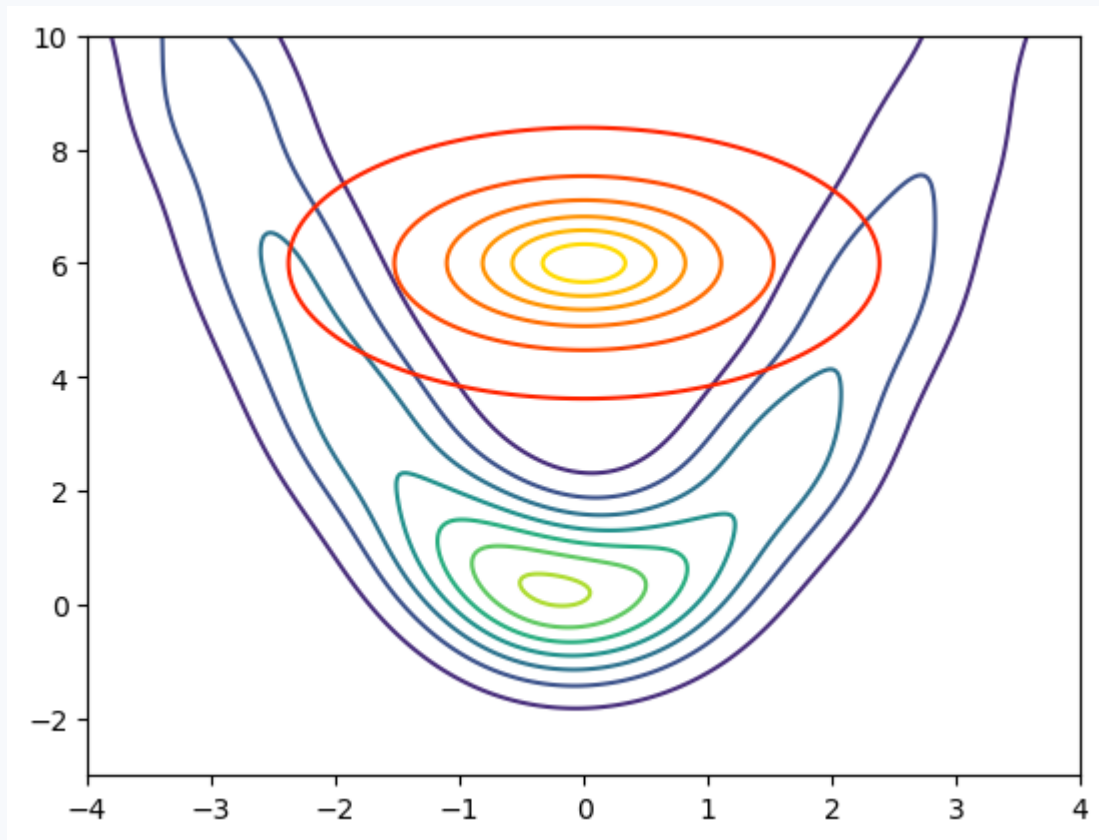
$$\begin{aligned}\mathbb{E}_{z \sim p(z)} f(z) &= \int f(z) p(z) dz = \\ &= \int \frac{f(z) p(z)}{q(z)} q(z) dz = \mathbb{E}_{z \sim q(z)} \frac{f(z) p(z)}{q(z)}\end{aligned}$$

$$\mathbb{E}_{z \sim q(z)} \frac{f(z) p(z)}{q(z)} \sim \frac{1}{N} \sum_{i=1}^N \frac{f(z_i) p(z_i)}{q(z_i)}, \quad z_i \sim q(z)$$



Estimation gets less noisy

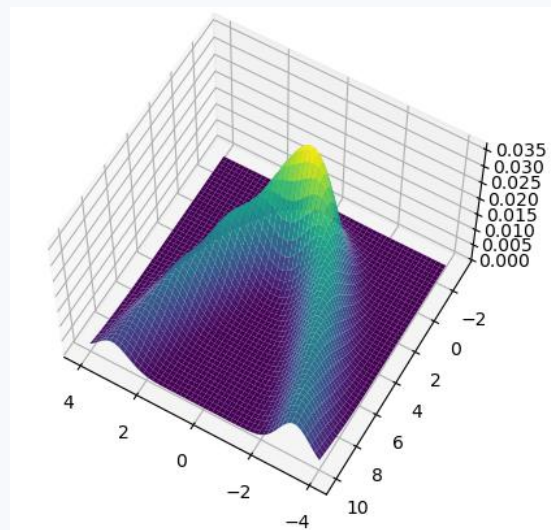
Importance sampling – example



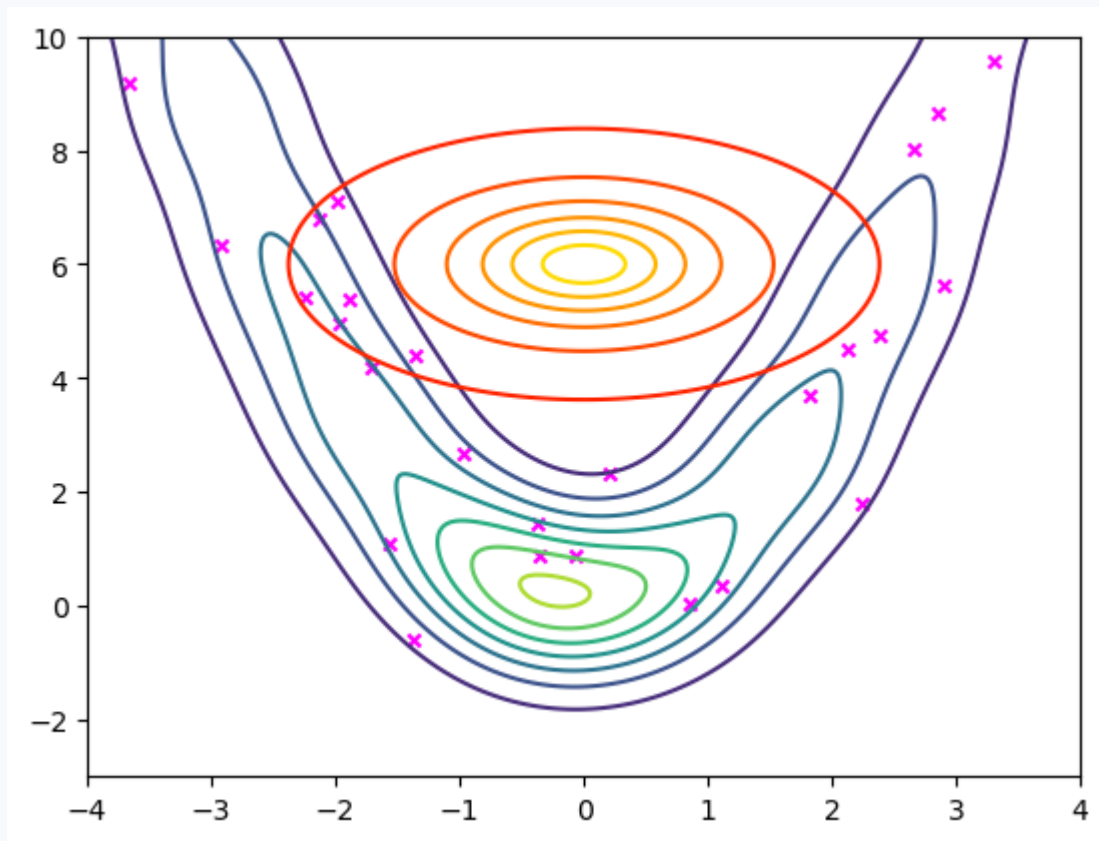
Distribution $p(z)$

Function $f(z)$

$$\mathbb{E}_{z \sim p(z)} f(z) = ?$$



Importance sampling – example



Distribution $p(z)$

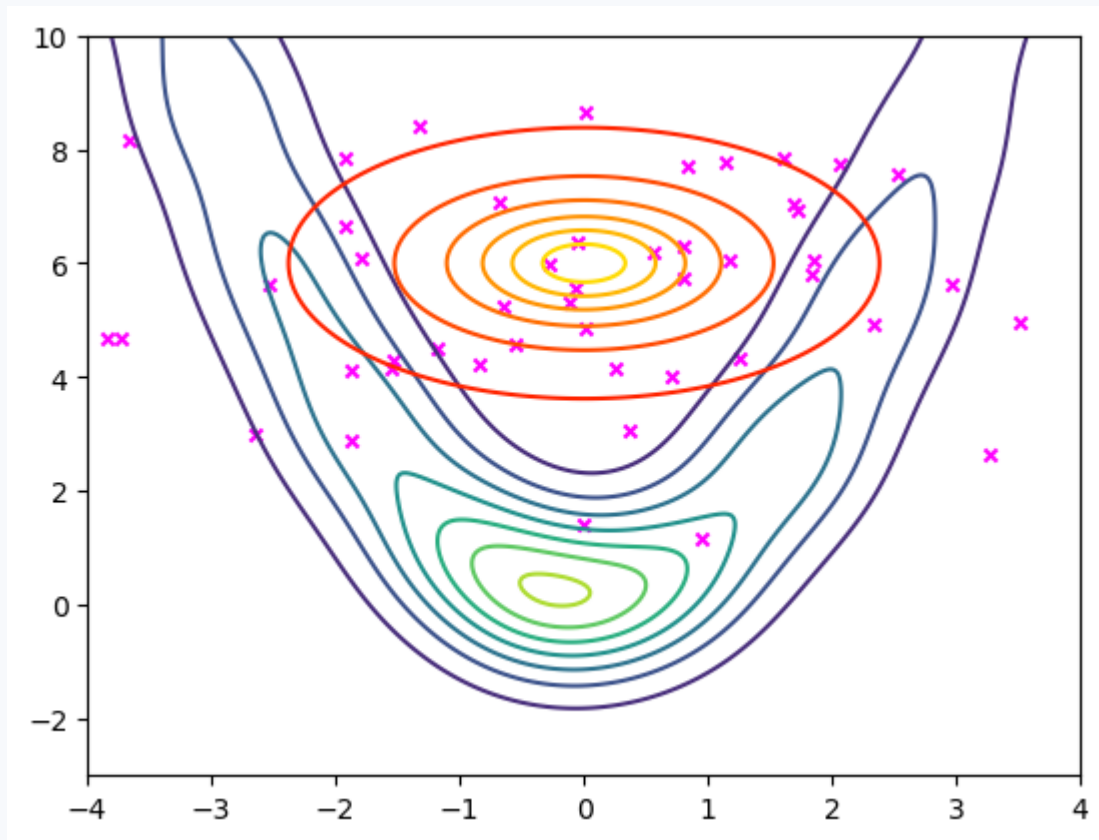
Function $f(z)$

$$\mathbb{E}_{z \sim p(z)} f(z) = ?$$

Monte Carlo:

$$\frac{1}{N} \sum_{i=1}^N f(z_i), \\ z_i \sim p(z)$$

Importance sampling – example



Distribution $p(z)$

Function $f(z)$

$$\mathbb{E}_{z \sim p(z)} f(z) = ?$$

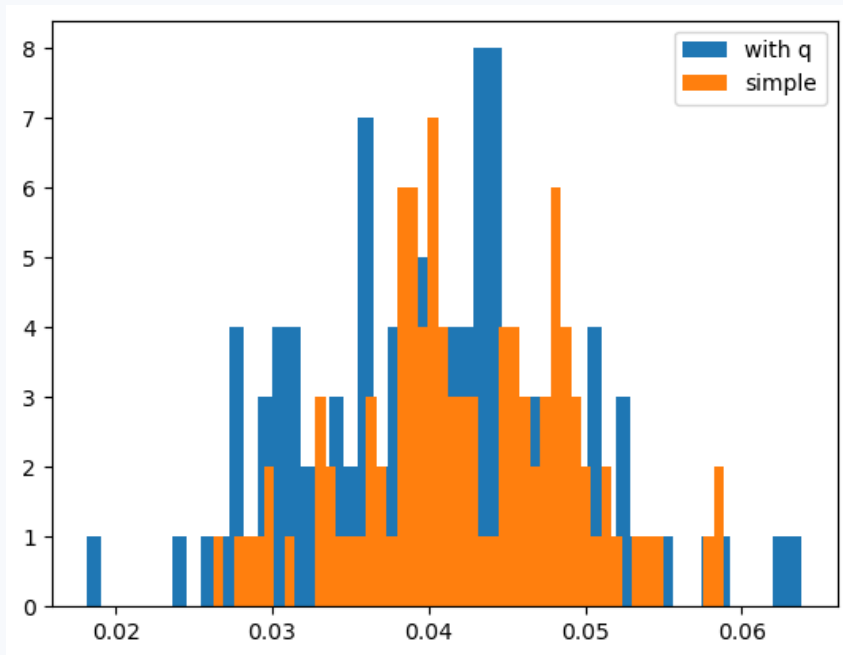
Importance sampling:

$$\frac{1}{N} \sum_{i=1}^N \frac{f(z_i) p(z_i)}{q(z_i)},$$

$z_i \sim q(z)$

Importance sampling – example

Importance sampling vs simple



Distribution $p(z)$

Function $f(z)$

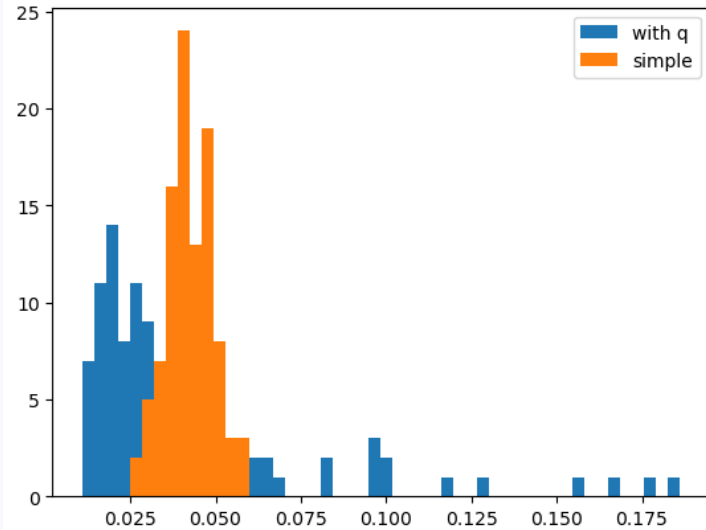
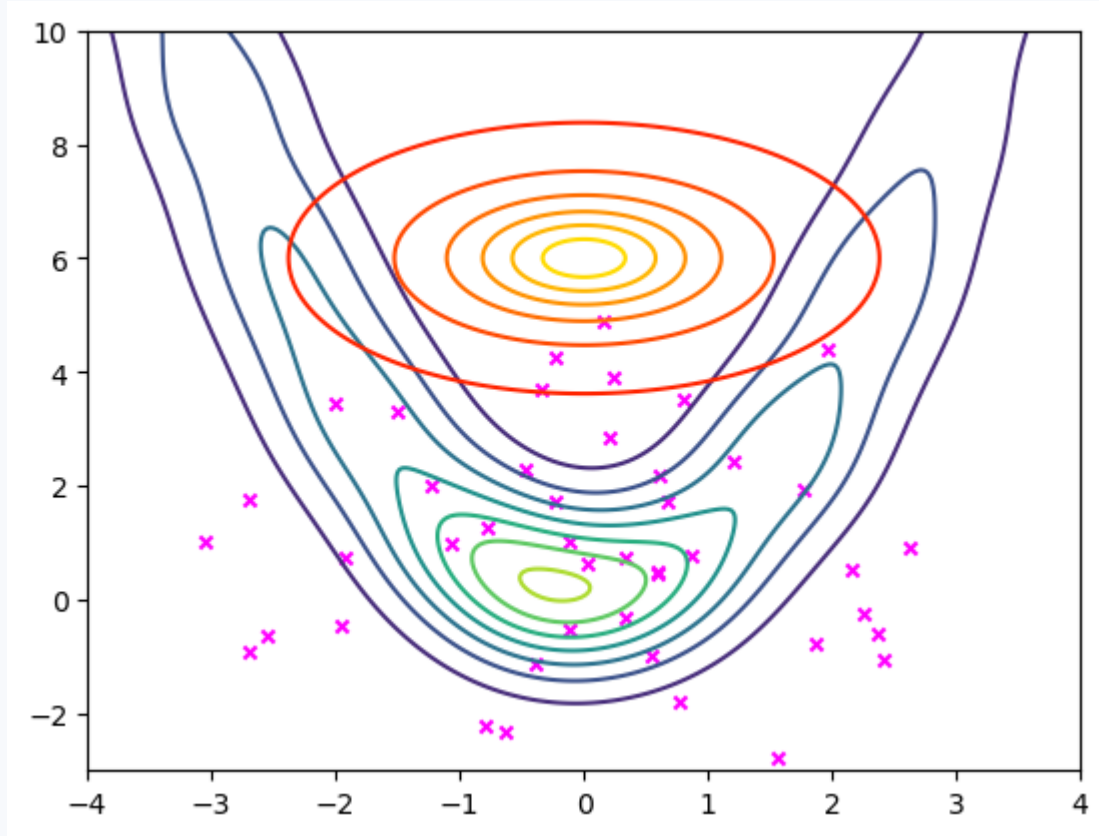
$$\mathbb{E}_{z \sim p(z)} f(z) = ?$$

Importance sampling:

$$\frac{1}{N} \sum_{i=1}^N \frac{f(z_i) p(z_i)}{q(z_i)},$$

$z_i \sim q(z)$

Importance sampling – another example



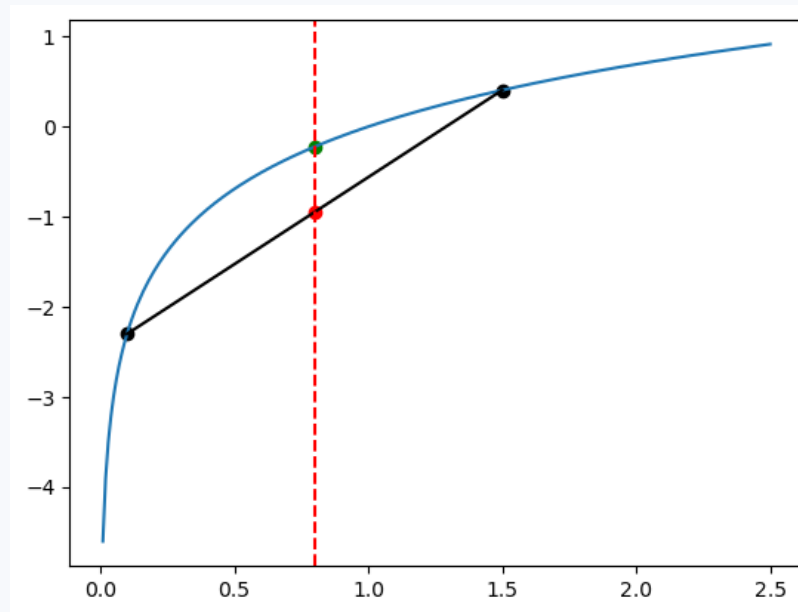
Jensen's inequality for log

Logarithm function is convex

$$\log\left(\frac{x_1 + x_2}{2}\right) \geq \frac{\log x_1 + \log x_2}{2}$$

More generally:

$$\log \mathbb{E} p(z) \geq \mathbb{E} \log p(z)$$



KL-divergence

If we have two distributions $p(x)$ and $q(x)$, their KL-divergence is:

$$D_{\text{KL}}(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_{x \sim p(x)} \log \frac{p(x)}{q(x)}$$

1. Asymmetric: $D_{\text{KL}}(q \parallel p) \neq D_{\text{KL}}(p \parallel q)$
2. $D_{\text{KL}}(q \parallel p) \geq 0$
3. $D_{\text{KL}}(q \parallel p) = 0$ only if p and q are the same
4. $D_{\text{KL}}(p_1 \parallel p_3) \leq D_{\text{KL}}(p_1 \parallel p_2) + D_{\text{KL}}(p_2 \parallel p_3)$

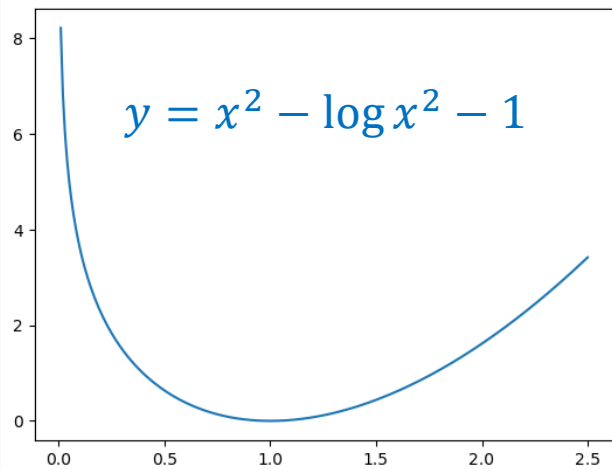
KL-divergence

If we have two distributions $p(x)$ and $q(x)$, their KL-divergence is:

$$D_{\text{KL}}(q \parallel p) = \int q(x) \log \frac{q(x)}{p(x)} dx = \mathbb{E}_{x \sim q(x)} \log \frac{q(x)}{p(x)}$$

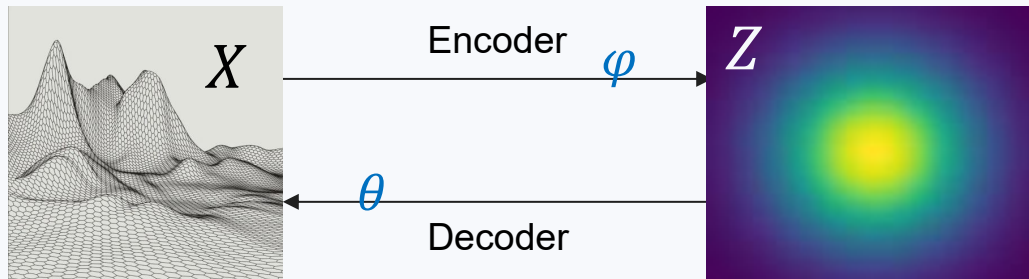
Example. $q = \mathcal{N}(\mu, \text{diag}(\sigma_1^2, \dots, \sigma_d^2))$, $p = \mathcal{N}(0, \mathbb{I})$,

$$D_{\text{KL}}(q \parallel p) = \frac{1}{2} \sum_{i=1}^d (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1)$$



Loss for VAE

VAE, a reminder



Write $q_{\varphi}(z | x)$ instead of $q(z | x, \varphi)$
to save some space

Encoder: $q_{\varphi}(z | x)$ with params φ .

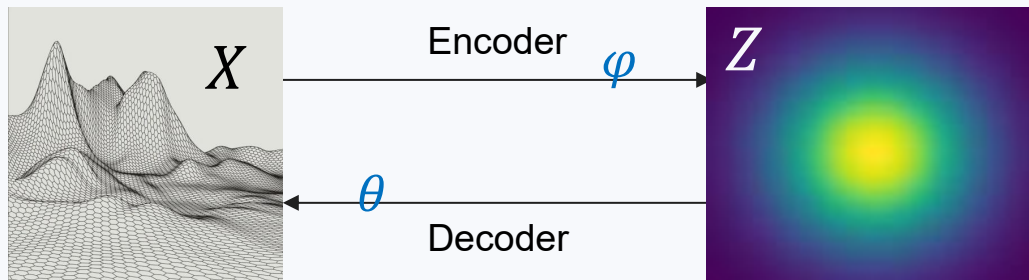
Decoder: $p_{\theta}(x | z)$ with params θ .

Latents have prior distribution:

$$p_{\theta}(z) = \mathcal{N}(z | 0, \mathbb{I})$$

The task is to estimate φ, θ

VAE, a reminder



Encoder: $q_{\varphi}(z | x)$ with params φ .

Decoder: $p_{\theta}(x | z)$ with params θ .

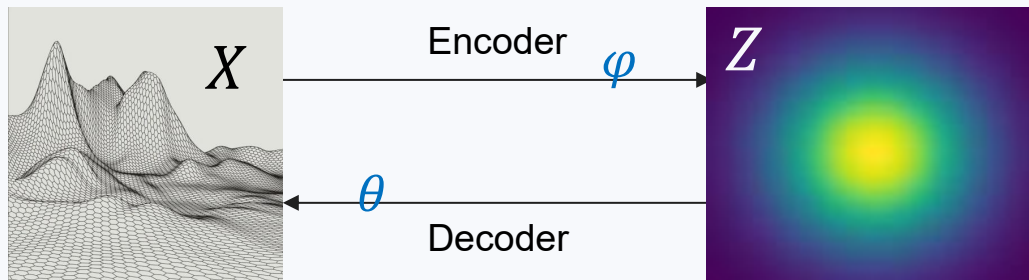
Sampling:

1. Sample z from $p_{\theta}(z) = \mathcal{N}(z | 0, \mathbb{I})$

2. Sample x from

$$\begin{aligned} p_{\theta}(x | z) \\ &= \mathcal{N}(x | \mu_{\theta}(z), \sigma^2 \mathbb{I}) \\ &= \mu_{\theta}(z) + \mathcal{N}(x | 0, \sigma^2 \mathbb{I}) \end{aligned}$$

VAE, a reminder



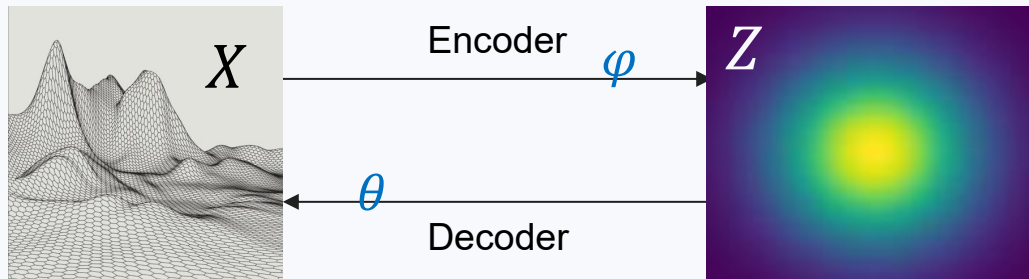
Encoder: $q_{\varphi}(z | x)$ with params φ .

Decoder: $p_{\theta}(x | z)$ with params θ .

Encoder:

$$q_{\varphi}(z | x) = \mathcal{N}\left(z \mid \begin{matrix} \mu_{\varphi}(x), \\ \text{diag}(\sigma_{1,\varphi}^2(x), \dots, \sigma_{d,\varphi}^2(x)) \end{matrix}\right)$$

VAE, a reminder



Encoder: $q_{\varphi}(z | x)$ with params φ .

Decoder: $p_{\theta}(x | z)$ with params θ .

Latents have prior distribution:

$$p_{\theta}(z) = \mathcal{N}(z | 0, \mathbb{I})$$

The task is to estimate φ, θ

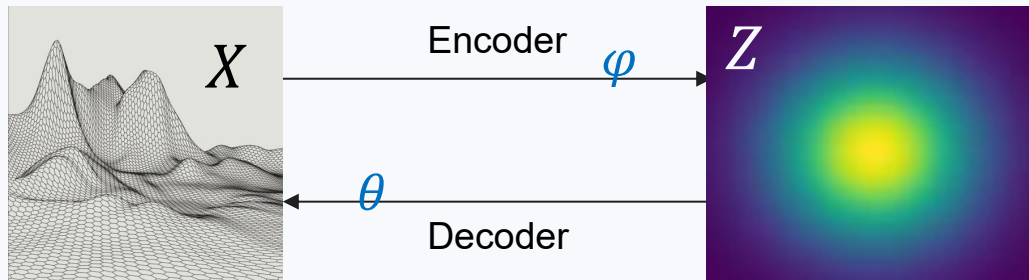
We optimize:

$$\log p_{\theta}(X) = \sum_i \log p_{\theta}(x_i), X = (x_1, x_2, \dots) - \text{dataset}$$

$$\log p_{\theta}(x) = \log \int p_{\theta}(x | z) p_{\theta}(z) dz \rightarrow \max_{\theta}$$

VAE, let's do some magic

$$\log p_{\theta}(x) \rightarrow \max_{\theta}$$



$$\log p_{\theta}(x) = \log \int p_{\theta}(x | z) p_{\theta}(z) dz = \log \int \frac{p_{\theta}(x | z) p_{\theta}(z)}{q_{\phi}(z | x)} q_{\phi}(z | x) dz =$$

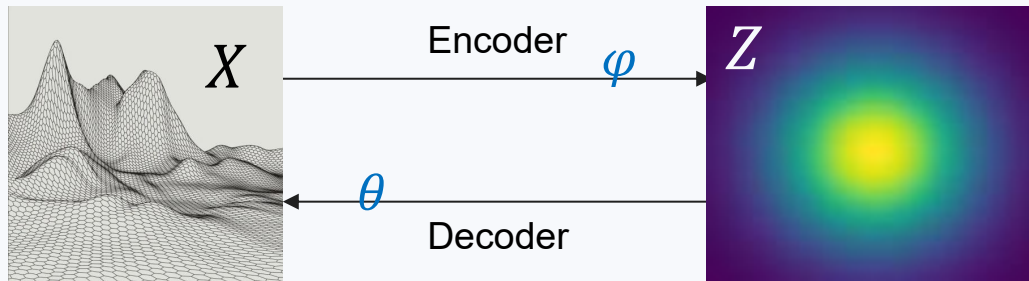
$$\mathbb{E}_{z \sim p_{\theta}(z)} p(x | z)$$

Importance sampling:
 $p_{\theta}(z) \rightarrow q_{\phi}(z | x)$

$$\mathbb{E}_{z \sim q_{\phi}(z | x)} \frac{p(x | z) p_{\theta}(z)}{q_{\phi}(z | x)}$$

VAE, let's do some magic

$$\log p_{\theta}(x) \rightarrow \max_{\theta}$$

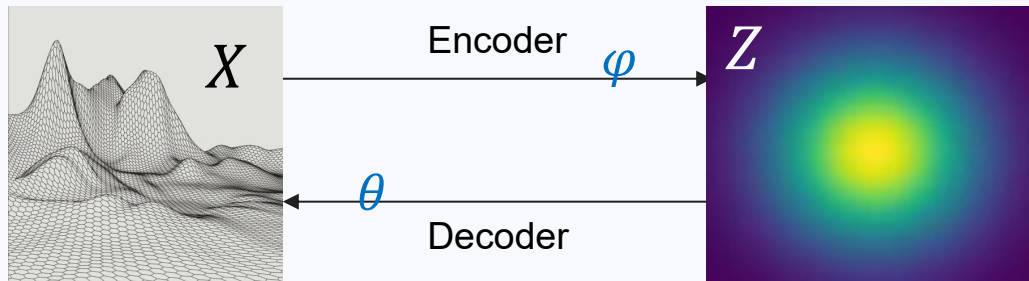


$$\begin{aligned}\log p_{\theta}(x) &= \log \int p_{\theta}(x | z) p_{\theta}(z) dz = \log \int \frac{p_{\theta}(x | z) p_{\theta}(z)}{q_{\phi}(z | x)} q_{\phi}(z | x) dz = \\ &= \log \mathbb{E}_{z \sim q_{\phi}(z | x)} \frac{p_{\theta}(x | z) p_{\theta}(z)}{q_{\phi}(z | x)} \geq \mathbb{E}_{z \sim q_{\phi}(z | x)} \log \left(\frac{p_{\theta}(x | z) p_{\theta}(z)}{q_{\phi}(z | x)} \right)\end{aligned}$$

Jentzen's inequality

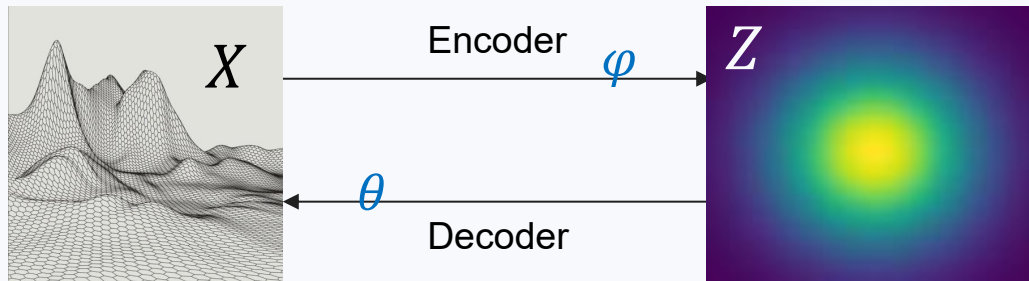
VAE, let's do some magic

$$\log p_{\theta}(x) \rightarrow \max_{\theta}$$



$$\begin{aligned}\log p_{\theta}(x) &= \log \int p_{\theta}(x | z) p_{\theta}(z) dz = \log \int \frac{p_{\theta}(x | z) p_{\theta}(z)}{q_{\phi}(z | x)} q_{\phi}(z | x) dz = \\ &= \log \mathbb{E}_{z \sim q_{\phi}(z | x)} \frac{p_{\theta}(x | z) p_{\theta}(z)}{q_{\phi}(z | x)} \geq \mathbb{E}_{z \sim q_{\phi}(z | x)} \log \left(\frac{p_{\theta}(x | z) p_{\theta}(z)}{q_{\phi}(z | x)} \right) = \\ &= \mathbb{E}_{z \sim q_{\phi}(z | x)} \log(p_{\theta}(x | z)) - \mathbb{E}_{z \sim q_{\phi}(z | x)} \log \frac{q_{\phi}(z | x)}{p_{\theta}(z)}\end{aligned}$$

VAE, let's do some magic



$$p_{\theta}(z) = \mathcal{N}(z \mid 0, \mathbb{I})$$

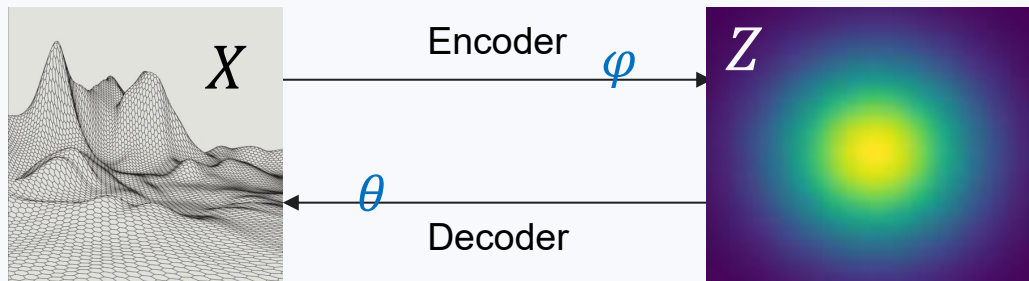
$$\log p_{\theta}(x) \rightarrow \max_{\theta}$$

$$\log p_{\theta}(x) = \log \int p_{\theta}(x \mid z) p_{\theta}(z) dz = \log \int \frac{p_{\theta}(x \mid z) p_{\theta}(z)}{q_{\phi}(z \mid x)} q_{\phi}(z \mid x) dz =$$

$$= \underbrace{\mathbb{E}_{z \sim q_{\phi}(z \mid x)} \log(p_{\theta}(x \mid z))}_{-(\text{Reconstruction loss})} - \underbrace{\mathbb{E}_{z \sim q_{\phi}(z \mid x)} \log \frac{q_{\phi}(z \mid x)}{p_{\theta}(z)}}_{\text{KL-divergence}} = \mathcal{D}_{\text{KL}}(q_{\phi}(z \mid x) \parallel p_{\theta}(z))$$

VAE, let's do some magic

$$p_{\theta}(z) = \mathcal{N}(z \mid 0, \mathbb{I})$$



$$q_{\phi}(z \mid x) = \mathcal{N}\left(z \mid \begin{matrix} \mu_{\phi}(x), \\ \text{diag}(\sigma_{\phi,1}^2(x), \dots, \sigma_{\phi,d}^2(x)) \end{matrix}\right)$$

$$\log p_{\theta}(x) = \mathbb{E}_{z \sim q_{\phi}(z|x)} \log(p_{\theta}(x \mid z)) - \mathbb{E}_{z \sim q_{\phi}(z|x)} \log \frac{q_{\phi}(z \mid x)}{p_{\theta}(z)}$$

Some details:

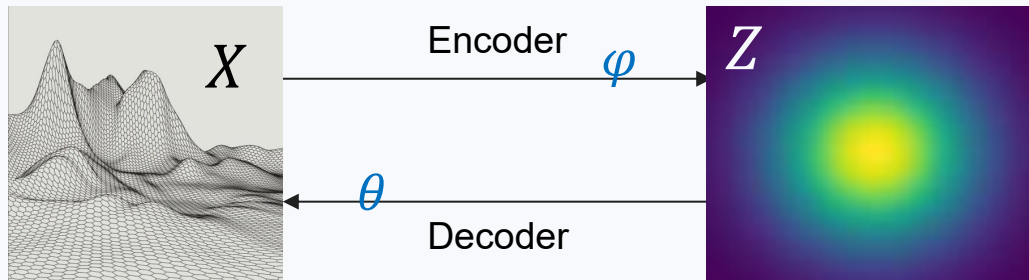
$$D_{\text{KL}}(q_{\phi}(z|x) \parallel p_{\theta}(z)) = \frac{1}{2} \sum_{i=1}^d \left(\mu_{\phi,i}^2(x) + \sigma_{\phi,i}^2(x) - \log \sigma_{\phi,i}^2(x) - 1 \right)$$

From $E(x)$

VAE, let's do some magic

$$p_{\theta}(z) = \mathcal{N}(z \mid 0, \mathbb{I})$$

$$q_{\phi}(z \mid x) = \mathcal{N}(z \mid \mu_{\phi}(x), \text{diag}(\sigma_1^2, \dots, \sigma_D^2))$$



$$\log p_{\theta}(x) = \mathbb{E}_{z \sim q_{\phi}(z|x)} \log(p_{\theta}(x \mid z)) - \mathbb{E}_{z \sim q_{\phi}(z|x)} \log \frac{q_{\phi}(z \mid x)}{p_{\theta}(z)}$$

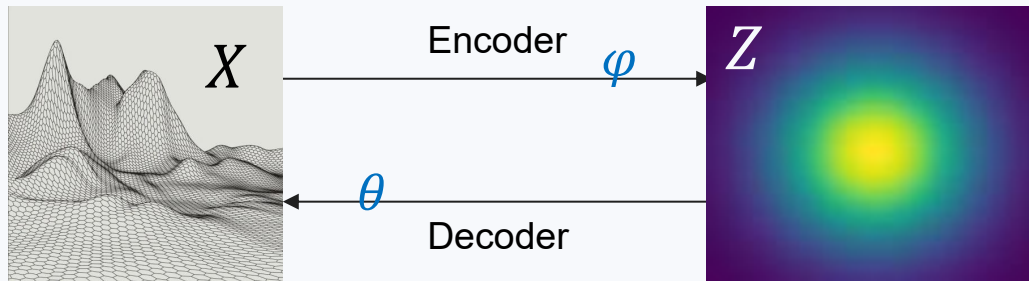
Some details:

$$\log p_{\theta}(x \mid z) = \log \mathcal{N}(x \mid \mu_{\theta}(z), \sigma^2 \mathbb{I}) = \sum_{j=1}^D \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x_j - \mu_{\theta,j})^2}{2\sigma^2} \right) \right)$$

VAE, let's do some magic

$$p_{\theta}(z) = \mathcal{N}(z \mid 0, \mathbb{I})$$

$$q_{\theta}(z \mid x) = \mathcal{N}(z \mid \mu_{\theta}(x), \sigma^2 \mathbb{I})$$

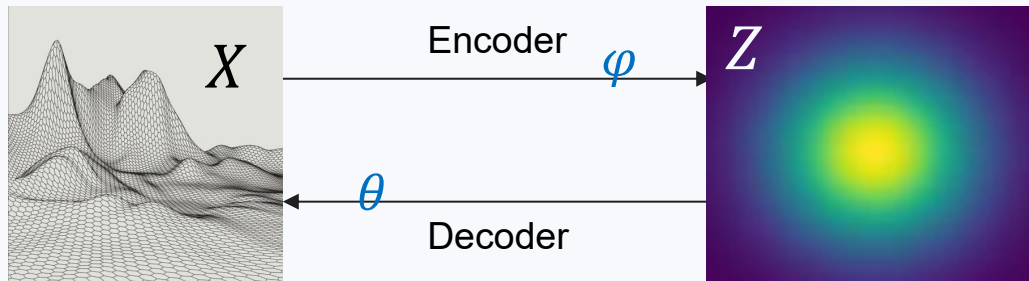


$$\log p_{\theta}(x) = \mathbb{E}_{z \sim q_{\phi}(z|x)} \log(p_{\theta}(x \mid z)) - \mathbb{E}_{z \sim q_{\phi}(z|x)} \log \frac{q_{\phi}(z \mid x)}{p_{\theta}(z)}$$

Some details:

$$\log p_{\theta}(x \mid z) = -\frac{D}{2} \log 2\pi - D \log \sigma - \frac{1}{2\sigma^2} \sum_{j=1}^D \left(x_j - \mu_{\theta,j}(z) \right)^2$$

VAE, let's do some magic



$$p_{\theta}(z) = \mathcal{N}(z \mid 0, \mathbb{I})$$

$$\log p_{\theta}(x \mid z) =$$

$$= \text{const} - \frac{1}{2\sigma^2} \|x - D(z)\|_2^2$$

$$\log p_{\theta}(x) = \mathbb{E}_{z \sim q_{\phi}(z|x)} \log(p_{\theta}(x \mid z)) - \mathbb{E}_{z \sim q_{\phi}(z|x)} \log \frac{q_{\phi}(z \mid x)}{p_{\theta}(z)}$$

Some details:

$$\mathbb{E}_{z \sim q_{\phi}(z|x)} \log(p_{\theta}(x \mid z)) = \text{const} - \frac{1}{2\sigma^2} \mathbb{E}_{z \sim q_{\phi}(z|x)} \|x - D(E(x))\|_2^2$$

That's reconstruction loss!

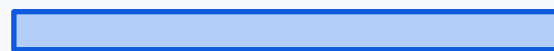
VAE modifications

β -VAE

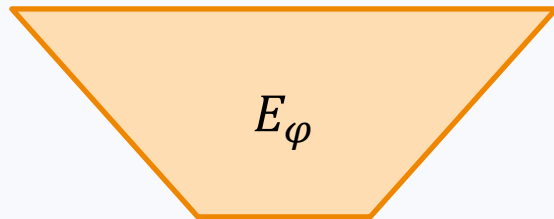
Loss:

$$\frac{1}{N} \sum_{i=1}^N \left\| D_{\theta} \left(E_{\varphi}(x_i) \right) - x_i \right\|^2 + \beta \cdot \text{KL} \left(\mathcal{N} \left(\mu_{\varphi}(x), \sigma_{\varphi}(x) \right) \parallel \mathcal{N}(0, I) \right)$$

To ensure that we can sample
 $\sim \mathcal{N}(0, I)$



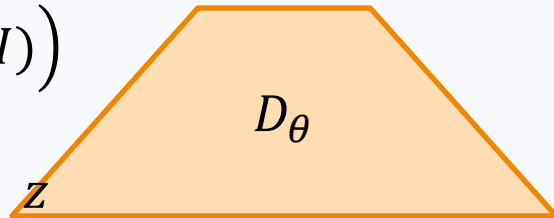
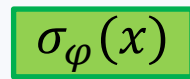
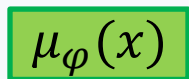
D-dimensional



Encoder



d-dimensional

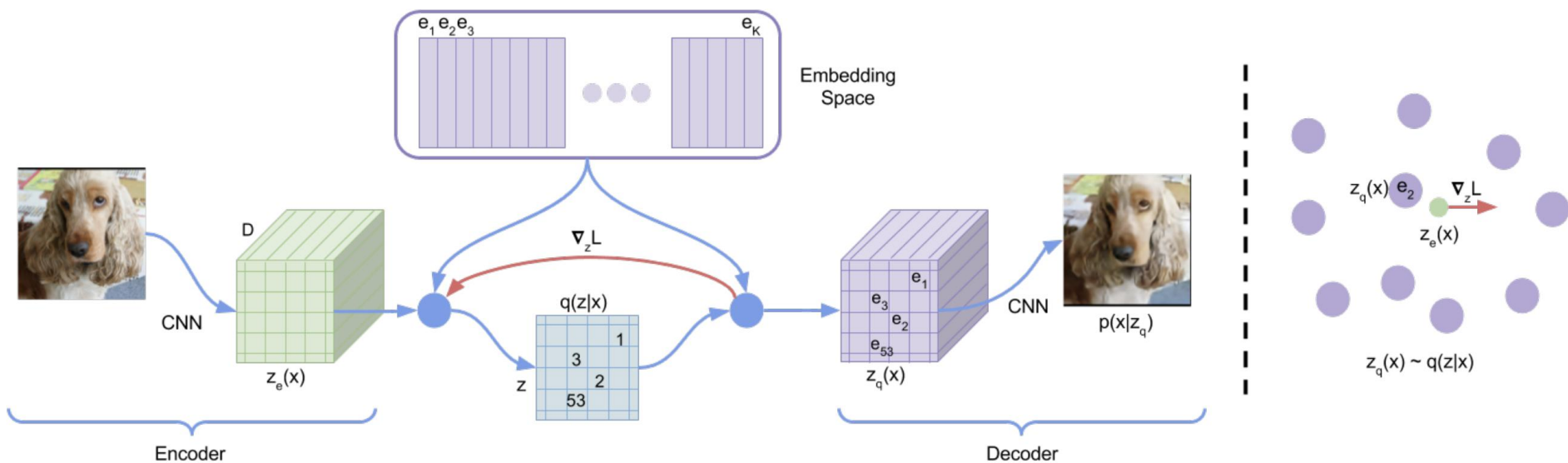


Decoder



D-dimensional

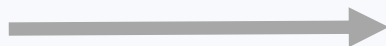
VQ-VAE



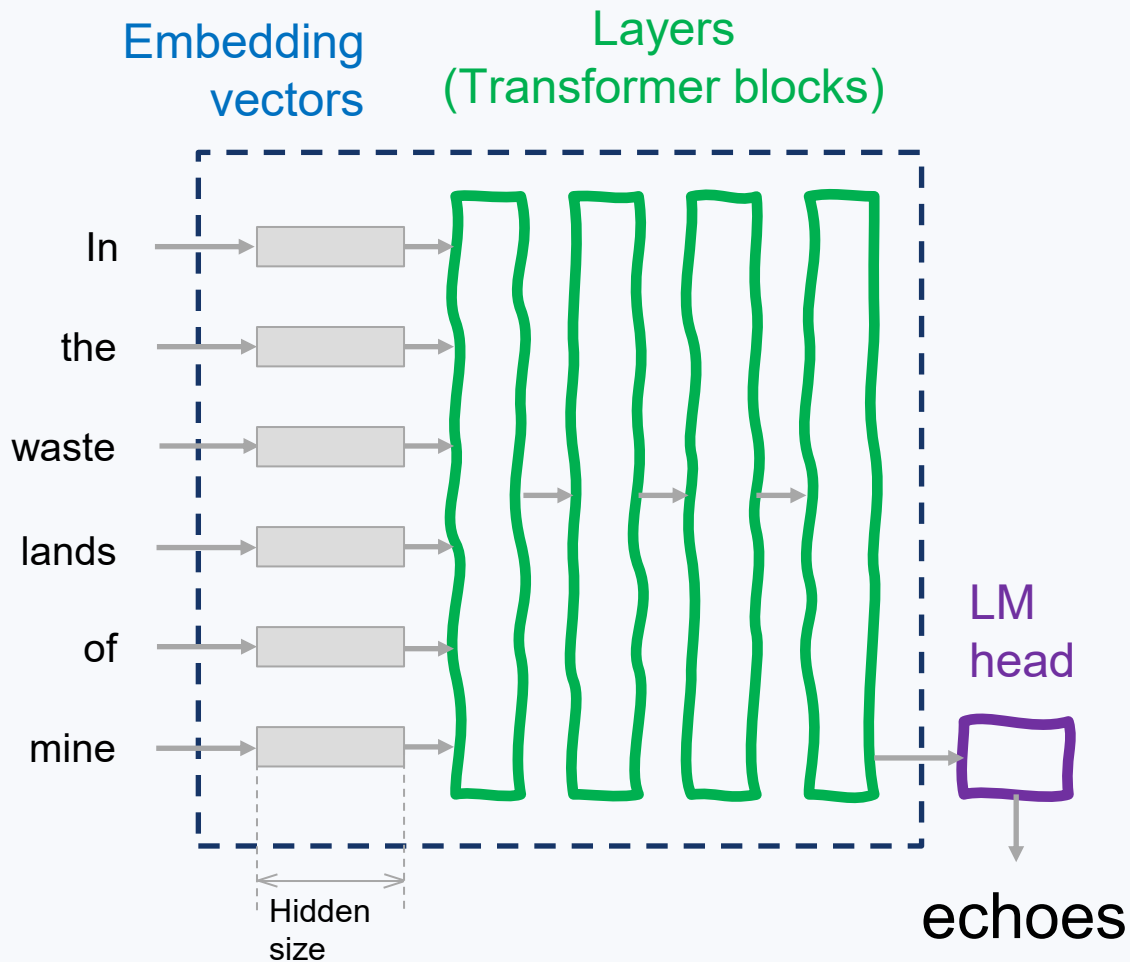
Working with text

The architecture of an LLM

In the
wastelands
of mine



Tokenization



Tokenization

Tokenization

Take a sentence: Darth Vader was born on Tatooine

Character-level: ["**BOS**", 'D', 'a', 'r', 't', 'h', ' ', 'V', 'a', 'd', 'e', 'r', ..., "**EOS**"]

Pros: A limited vocabulary

Cons: Too little information in each token

"**BOS**" = beginning of sequence; "**EOS**" = end of sequence

Tokenization

Take a sentence: Darth Vader was born on Tatooine

Character-level: ["**BOS**", 'D', 'a', 'r', 't', 'h', ' ', 'V', 'a', 'd', 'e', 'r', ..., "**EOS**"]

Pros: A limited vocabulary

Cons: Too little information in each token

"**BOS**" = beginning of sequence; "**EOS**" = end of sequence

Word-level: ["**BOS**", "Darth", "Vader", "was", "born", "on", ...]

Pros: Simple

Cons: Potentially infinite vocabulary (typos, neologisms,...). And what to do at test time with the tokens the model never saw during training?

Subword tokenization: byte pair encoding

Fix the dictionary size V

Take a large text corpus

Idea: capture

Start with characters **'a'**, **'b'**, ..., **'1'**, ..., **'\b'**, ...

Do, until you get V tokens:

- Find the most frequent pair of consecutive tokens. For example, **'w'+'e'**
- Glue them into a new token: **"we"**

You can choose to glue tokens across spaces, if you want.

"import pandas as pd" might be a good token

Examples (OpenAI's o200k_base tokenizer)

The quick brown fox jumps over the lazy dog.	['The', ' quick', ' brown', ' fox', ' jumps', ' over', ' the', ' lazy', ' dog', '.']
Быстрая бурая лисица перепрыгивает через ленивую собаку.	['Б', 'ы', 'стра', 'я', ' бу', 'ра', 'я', ' л', 'иси', 'ца', ' переп', 'рыг', 'ивает', ' через', ' лен', 'ив', 'ую', ' соб', 'аку', '.']
敏捷的棕色狐狸跳過那隻懶狗。	['敏', '捷', '的', '0xe6a3', '0x95', '色', '狐狸', '跳', '過', '那', '0xe99a', '0xbb', '0xe687', '0xb6', '狗', '。']
재빠른 갈색 여우가 게으른 개를 뛰어넘는다.	['재', '빠', '른', ' 갈', '색', ' 여', '우', '가', ' 게', '으', '른', ' 개', '를', ' 뛰', '어', '넘', '는다', '.']

Examples (OpenAI's o200k_base tokenizer)

The quick brown fox jumps over the lazy dog.	['ĠThe', 'Ġquick', 'Ġbrown', 'Ġfox', 'Ġjumps', 'Ġover', 'Ġthe', 'Ġlazy', 'Ġdog', '.']
Быстрая бурая лисица перепрыгивает через ленивую собаку.	['ĠБ', 'ы', 'стра', 'я', 'Ġбу', 'ра', 'я', 'Ġл', 'иси', 'ца', 'Ġпереп', 'рыг', 'ивает', 'Ġчерез', 'Ġлен', 'ив', 'ую', 'Ġсоб', 'аку', '.']
敏捷的棕色狐狸跳過那隻懶狗。	['敏', '捷', '的', '0xe6a3', '0x95', '色', '狐狸', '跳', '過', '那', '0xe99a', '0xbb', '0xe687', '0xb6', '狗', '。']
재빠른 갈색 여우가 게으른 개를 뛰어넘는다.	['재', '빠', '른', '갈', '색', '여', '우', '가', '게', '으', '른', '개', '를', '뛰', '어', '넘', '는다', '.']

Examples

`\textbf{Definition.}`

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

We say that f is `\textbf{$\mu$-strongly convex}` if there exists $\mu > 0$ such that

$$\begin{aligned} & f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \\ & \quad \frac{\mu}{2} \|y - x\|^2 \\ & \quad \text{for all } x, y \in \mathbb{R}^n. \end{aligned}$$

Equivalently, the function

$$\begin{aligned} & x \mapsto f(x) - \frac{\mu}{2} \|x\|^2 \\ & \text{is convex.} \end{aligned}$$

[`\t`, `'ext'`, `'bf'`, `{`, `'Definition'`, `'.'`, `}`, `\n`, `'Let'`, `' '`, `'$'`, `'f'`, `'.'`, `' '`, `'\l`, `'math'`, `'bb'`, `{`, `'R'`, `'\}`, `'^'`, `'n'`, `' '`, `'\to'`, `'\l`, `'math'`, `'bb'`, `{`, `'R'`, `'\}`, `'$'`, `'.'`, `\n`, `'We'`, `' say'`, `' that'`, `' '`, `'$'`, `'f'`, `'$'`, `' is'`, `' '`, `'\t`, `'ext'`, `'bf'`, `{`, `'$'`, `'\l`, `'mu'`, `'$'`, `'-'`, `'strong'`, `'ly'`, `' convex'`, `'\}`, `' if'`, `' there'`, `' exists'`, `' '`, `'\l`, `'mu'`, `'>`, `' '`, `'0'`, `'$'`, `' such'`, `' that'`, `'\l`, `'['`, `'f'`, `'(y'`, `' )'`, `'\l`, `'ge'`, `'f'`, `'(x'`, `' )'`, `' +'`, `'\l`, `'l'`, `'angle'`, `'\n`, `'abla'`, `'f'`, `'(x'`, `' )'`, `'\l`, `'y'`, `'-x'`, `'\r'`, `'angle'`, `' +'`, `' '`, `'\x0c'`, `'rac'`, `{`, `'\l`, `'mu'`, `'\}`, `'2'`, `'\l`, `'['`, `'y'`, `'-x'`, `'\l`, `'['`, `'^'`, `'2'`, `'\l`, `'qqu'`, `'ad'`, `' '`, `'\x0c'`, `'or'`, `'all'`, `'x'`, `'y'`, `'\l`, `'in'`, `'\l`, `'math'`, `'bb'`, `{`, `'R'`, `'\}`, `'^'`, `'n'`, `'\l`, `'\n`, `'Equ'`, `'ival'`, `'ently'`, `' '`, `' the'`, `' function'`, `'\l`, `'['`, `'x'`, `'\l`, `'map'`, `'sto'`, `'f'`, `'(x'`, `' )'`, `' -'`, `' '`, `'\x0c'`, `'rac'`, `{`, `'\l`, `'mu'`, `'\}`, `'2'`, `'\l`, `'['`, `'x'`, `'\l`, `'['`, `'^'`, `'2'`, `'\l`, `'['`, `' is'`, `' convex'`, `'.'`]

Examples

Definition.

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$. We say that f is **strongly convex** with parameter $\mu > 0$ if

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \mu/2 |y - x|^2$$

for all $x, y \in \mathbb{R}^n$.

Equivalently,

$$x \mapsto f(x) - \frac{\mu}{2} |x|^2$$

is convex.

['0xf09d90', '0x83', '0xf09d90', '0x9e', '0xf09d90', '0x9f',
'0xf09d90', '0xa2', '0xf09d90', '0xa7', '0xf09d90', '0xa2',
'0xf09d90', '0xad', '0xf09d90', '0xa2', '0xf09d90', '0xa8',
'0xf09d90', '0xa7', '.\n', 'Let', 'f', ':', '0x20e284', '0x9d',
'0xe281', '0xbf', '→', '0x20e284', '0x9d', ':', 'We', 'say',
'that', 'f', 'is', '', '0xf09d90', '0xac', '0xf09d90', '0xad',
'0xf09d90', '0xab', '0xf09d90', '0xa8', '0xf09d90', '0xa7',
'0xf09d90', '0xa0', '0xf09d90', '0xa5', '0xf09d90', '0xb2', '',
'0xf09d90', '0x9c', '0xf09d90', '0xa8', '0xf09d90', '0xa7',
'0xf09d90', '0xaf', '0xf09d90', '0x9e', '0xf09d90', '0xb1',
'with', 'parameter', 'μ', '>', '', '0', 'if', '\n\n', 'f', '(y,)', '≥', 'f',
'(x,)', '+', '0x20e2', '0x9f', '0xa8', '0xe288', '0x87', 'f', '(x',
'),', 'y', '-', 'x', '0xe2', '0x9f', '0xa9', '+', 'μ', '/', '2', '|', 'y',
'-', 'x', '|', '2', '\n\n', 'for', 'all', 'x', ',', 'y', '0x20e288', '0x88',
'0x20e284', '0x9d', '0xe281', '0xbf', '.\n\n', 'Equ', 'ival', 'ently',
' ,\n\n', 'x', '0x20e286', '0xa6', 'f', '(x,)', '-', '', '\x0c', 'rac',
'{', 'μ', '}', '2', '}', '|', 'x', '|', '2', '\n\n', 'is', 'convex', '.']

Examples

Definition.

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$. We say that f is **strongly convex** with parameter $\mu > 0$ if

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \mu/2 |y - x|^2$$

for all $x, y \in \mathbb{R}^n$.

Equivalently,

$$x \mapsto f(x) - \frac{\mu}{2} |x|^2$$

is convex.

['0xf09d90', '0x83', '0xf09d90', '0x9e', '0xf09d90', '0x9f',
'0xf09d90', '0xa2', '0xf09d90', '0xa7', '0xf09d90', '0xa2',
'0xf09d90', '0xad', '0xf09d90', '0xa2', '0xf09d90', '0xa8',
'0xf09d90', '0xa7', '.\n', 'Let', 'f', ':', '0x20e284', '0x9d',
'0xe281', '0xbf', '→', '0x20e284', '0x9d', ':', 'We', 'say', '
that', 'f', 'is', '', '0xf09d90', '0xac', '0xf09d90', '0xad',
'0xf09d90', '0xab', '0xf09d90', '0xa8', '0xf09d90', '0xa7',
'0xf09d90', '0xa0', '0xf09d90', '0xa5', '0xf09d90', '0xb2', '',
'0xf09d90', '0x9c', '0xf09d90', '0xa8', '0xf09d90', '0xa7',
'0xf09d90', '0xaf', '0xf09d90', '0x9e', '0xf09d90', '0xb1', '
with', 'parameter', 'μ', '>', '', '0', 'if', '\n\n', 'f', '(y,)', '≥', 'f',
'(x,)', '+', '0x20e2', '0x9f', '0xa8', '0xe288', '0x87', 'f', '(x',
)', 'y', '-', 'x', '0xe2', '0x9f', '0xa9', '+', 'μ', '/', '2', '|', 'y', '
-', 'x', '|', '2', '\n\n', 'for', 'all', 'x', ',', 'y', '0x20e288', '0x88',
'0x20e284', '0x9d', '0xe281', '0xbf', '.\n\n', 'Equ', 'ival', 'ently',
, '\n\n', 'x', '0x20e286', '0xa6', 'f', '(x,)', '-', '', '\x0c', 'rac',
{', 'μ', '}', '2', '}', '|', 'x', '|', '2', '\n\n', 'is', 'convex', '.']

Examples

```
import torch
```

```
class SwishFn(torch.autograd.Function):
```

```
    @staticmethod
```

```
    def forward(ctx, x):
```

```
        ctx.save_for_backward(x)
```

```
        return x * torch.sigmoid(x)
```

```
    @staticmethod
```

```
    def backward(ctx, grad_output):
```

```
        (x,) = ctx.saved_tensors
```

```
        sig = torch.sigmoid(x)
```

```
        return grad_output * (sig + x * sig * (1 - sig))
```

```
x = torch.randn(6, requires_grad=True)
```

```
y = SwishFn.apply(x).sum()
```

```
y.backward()
```

```
print(x.grad)
```

```
['import', ' torch', '\\n\\n', 'class', ' Sw', 'ish', 'Fn',  
'(torch', '.aut', 'ograd', '.Function', '):\\n', ' ', '@',  
'staticmethod', '\\n', ' ', ' def', ' forward', '(ctx', ',', ' x',  
)\\n', ' ', ' ', ' return', ' x', '*', ' torch', '.sig', 'moid',  
'(x', ')\\n\\n', ' ', ' ', '@', 'staticmethod', '\\n', ' ', ' def', '  
backward', '(ctx', ',', ' grad', '_output', '):\\n', ' ', ' ',  
(, 'x', ',)', ' =', ' ctx', '.saved', '_t', 'ensors', '\\n', ' ',  
' sig', ' =', ' torch', '.sig', 'moid', '(x', ')\\n', ' ', ' ',  
return', ' grad', '_output', '*', ' (, 'sig', '+', ' x', '*', '  
sig', '*', ' (, '1', '-', ' sig', ')\\n\\n', 'x', ' =', ' torch',  
'randn', '(, '6', ',', ' requires', '_grad', '=True', ')\\n',  
'y', ' =', ' Sw', 'ish', 'Fn', '.apply', '(x', ').', 'sum', '()\\n',  
'y', '.backward', '()\\n', 'print', '(x', '.grad', ')]
```

Examples – different tokenizers

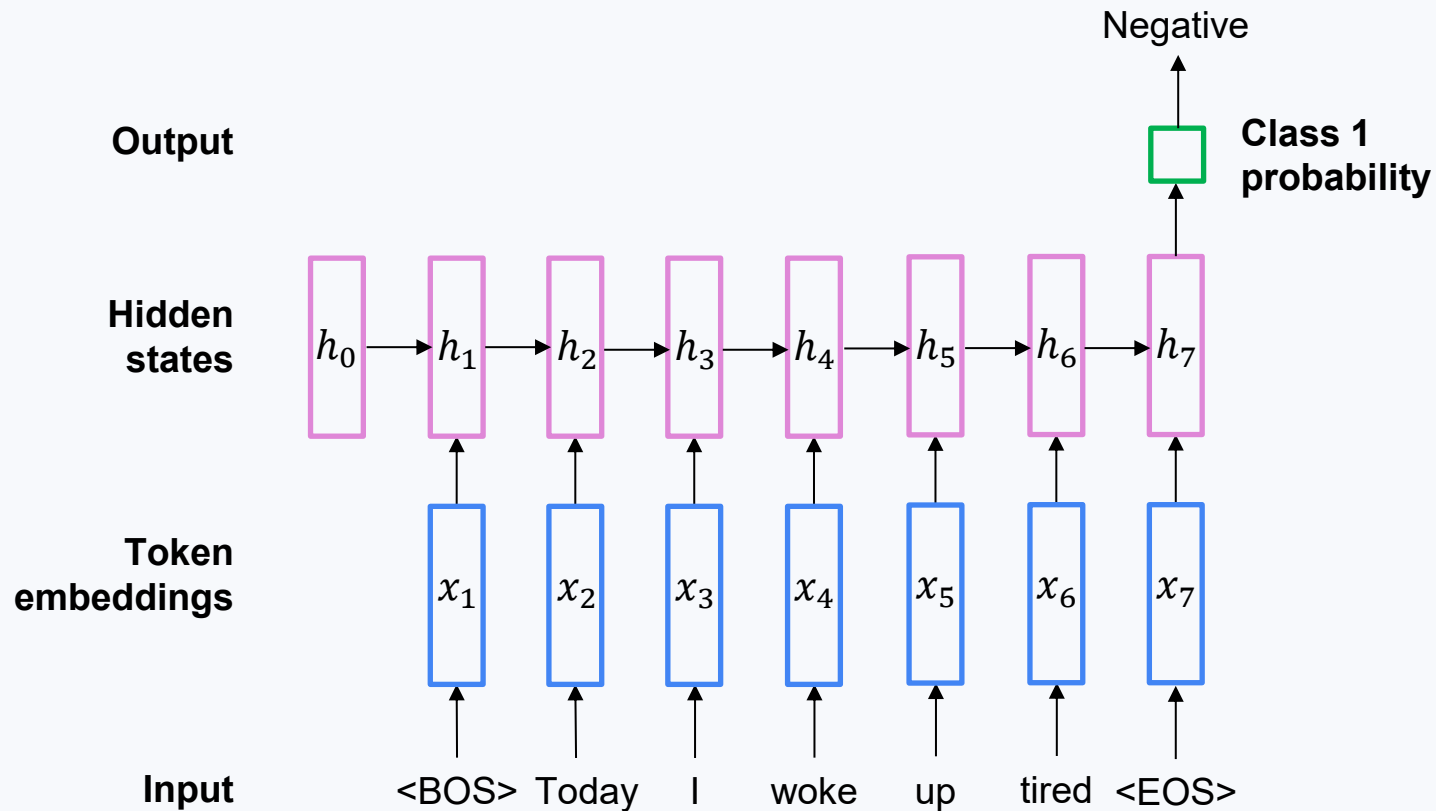
Original	Palimpsest, 裡買下了
OpenAI (o200k_base)	['Pal', 'im', 'ps', 'est', ',', '0x20e8a3', '0xa1', '買', '下', '了']
Qwen3.5-9B	['Pal', 'impse', 'st', ',', ',', '裡', '買', '下了']
DeepSeek-R1	['Pal', 'imps', 'est', ',', ',', '裡', '買', '下了']

Takeaways

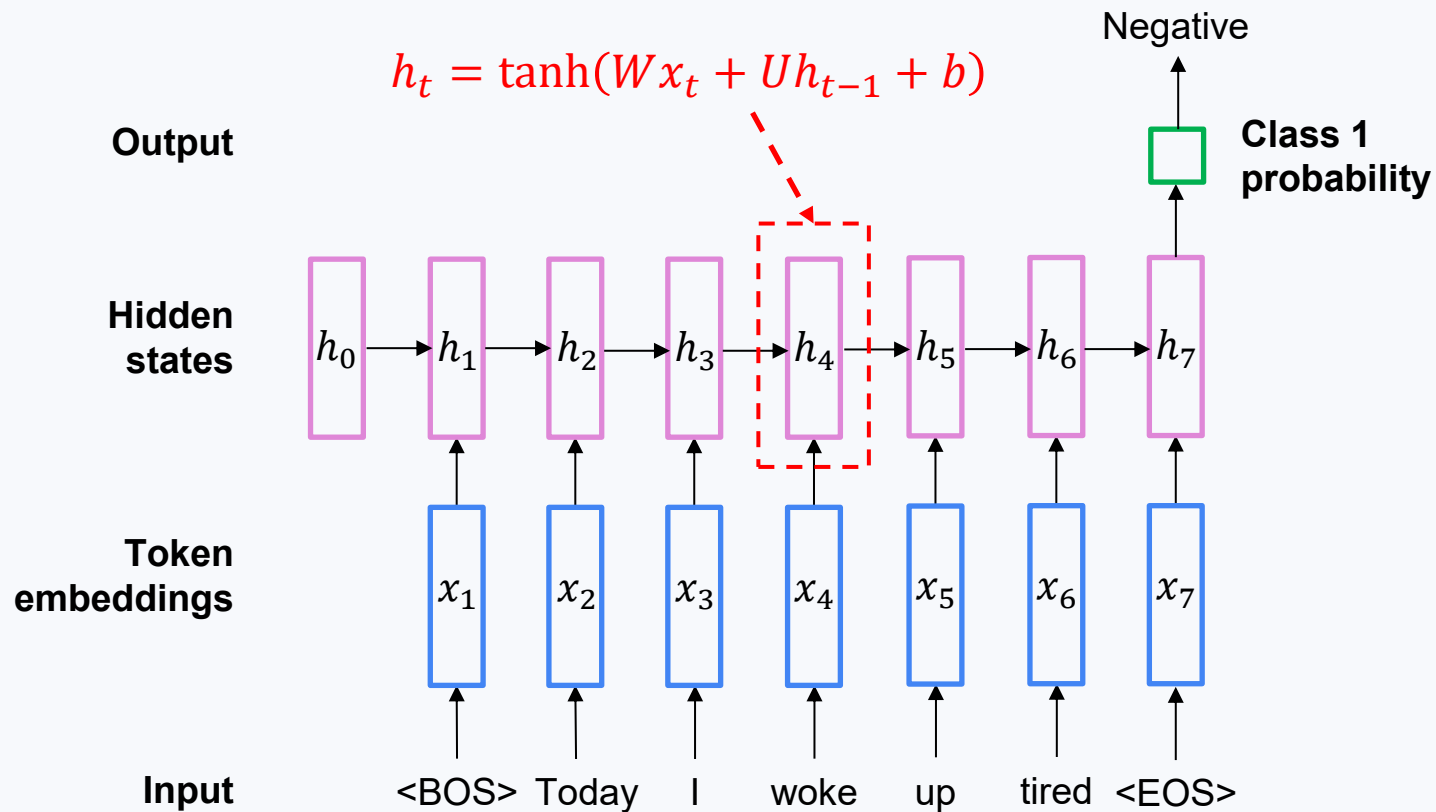
- Texts in less popular languages are shred to lesser tokens and give longer tokenized sequences
- Different models have different, non-compatible tokenizers

Recurrent networks

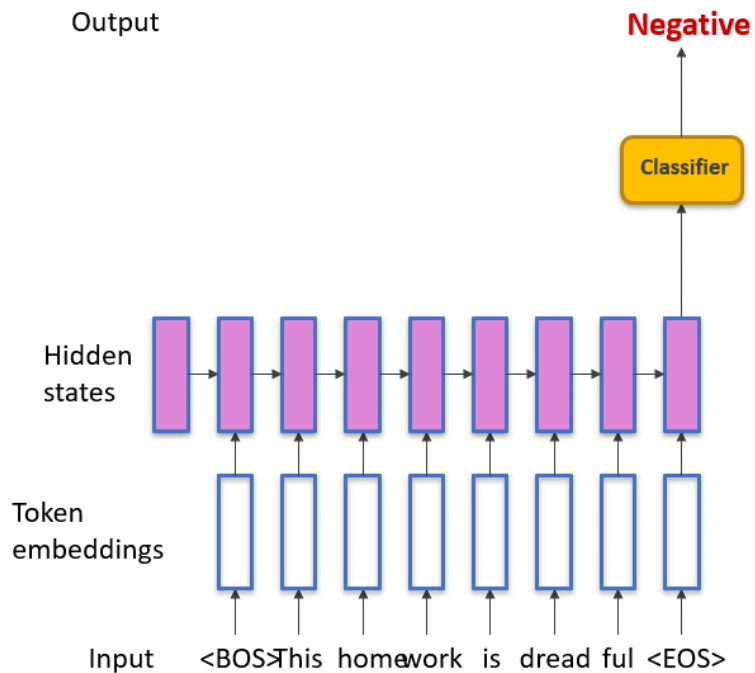
A recurrent network idea



A recurrent network idea



A recurrent network idea



Backpropagation

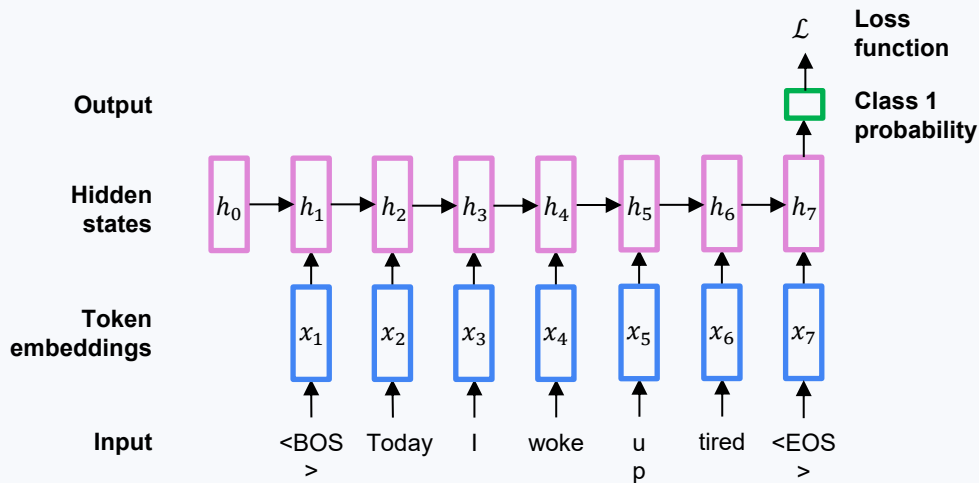
$$h_t = \tanh(Wx_t + Uh_{t-1} + b)$$

$$\frac{\partial \mathcal{L}}{\partial U} = \frac{\partial \mathcal{L}}{\partial h_7} \cdot \frac{dh_7}{dU} = \frac{\partial \mathcal{L}}{\partial h_7} \cdot \left(\frac{\partial h_7}{\partial U} + \frac{\partial h_7}{\partial h_6} \cdot \frac{dh_7}{dU} \right) = \dots$$

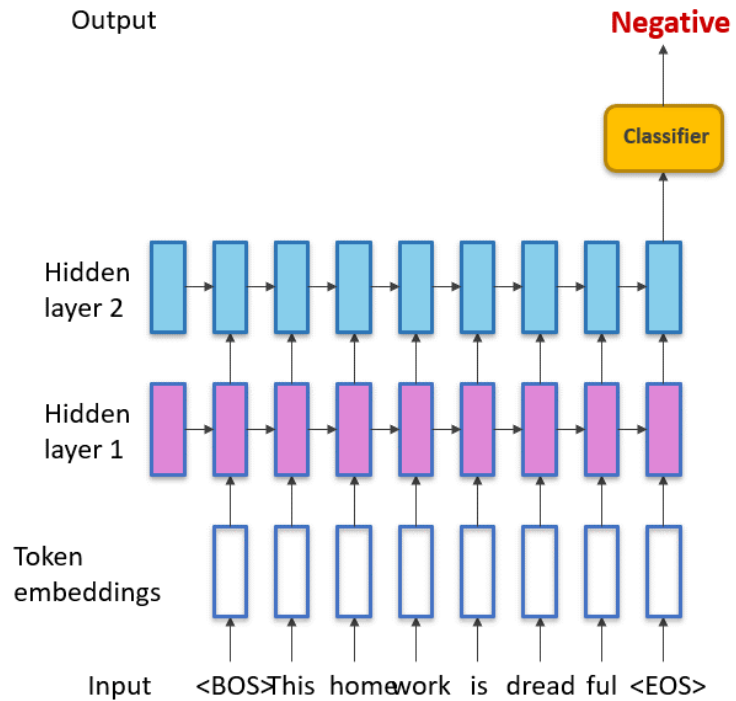
You'll have:

$$\frac{\partial h_7}{\partial h_6} \cdot \frac{\partial h_6}{\partial h_6} \cdot \dots \cdot \frac{\partial h_2}{\partial h_1}$$

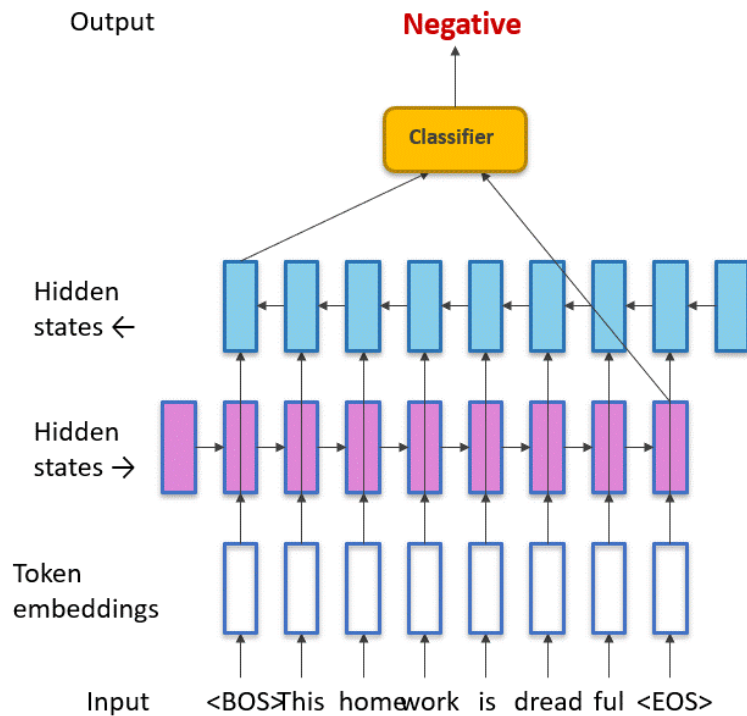
- Lots of derivatives of activations (they better be bounded)
- Lots of U^T (possibility of exploding or vanishing gradients)
- A popular technique was **gradient clipping**



Stack more layers!

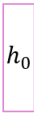


Bidirectional RNNs



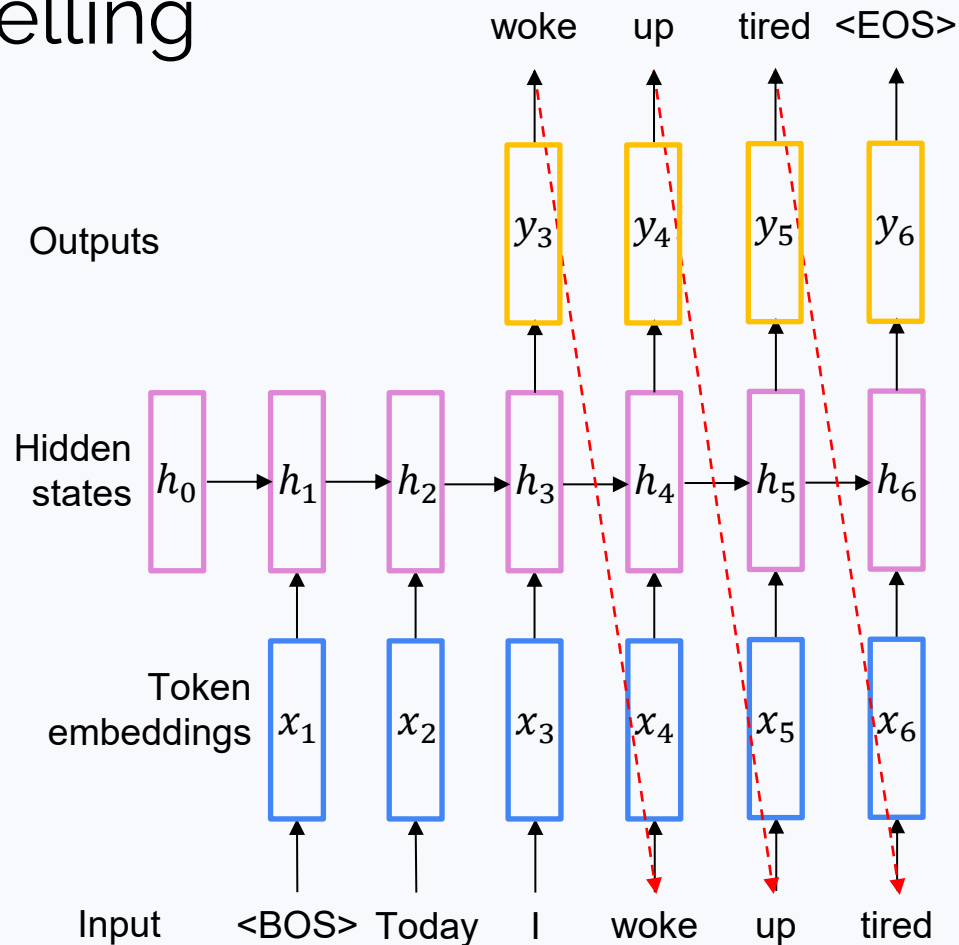
Language modelling with RNNs

Hidden
states

A vertical purple rectangular box containing the symbol h_0 .

Input <BOS> Today I

Language modelling with RNNs



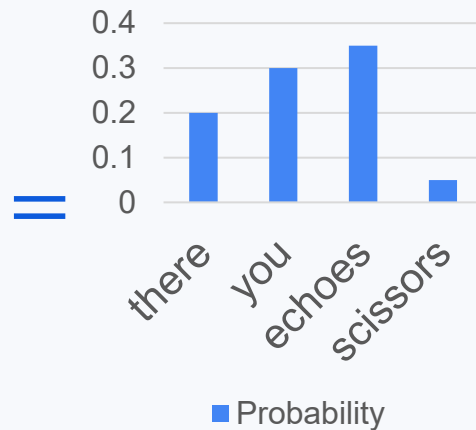
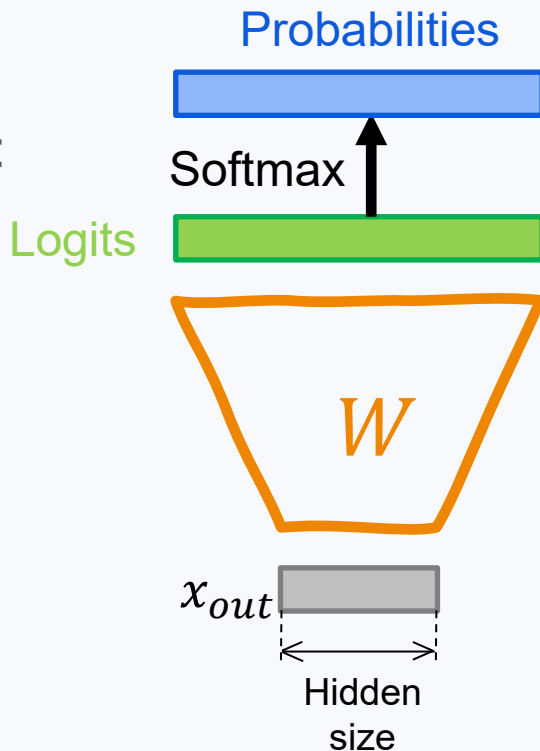
LM head

Basically, it is
a linear transformation:

$$\text{Logits} = x_{out} \cdot W$$

Probabilities

$$= \text{Softmax}(x_{out} \cdot W)$$



It's a Linear Classifier

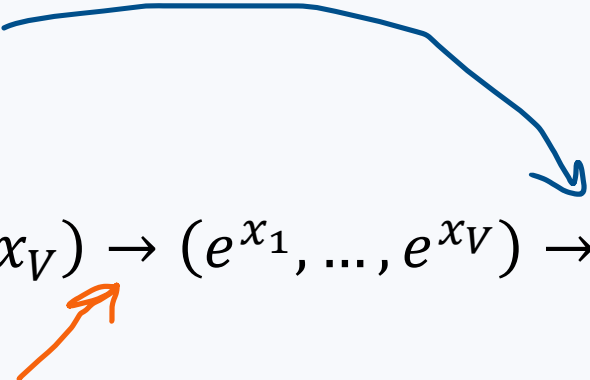
It takes Features
and outputs

Class Probabilities

Softmax

Turns **logits** (can be any numbers) into probabilities, that

- Are all non-negative,
- Sum to 1

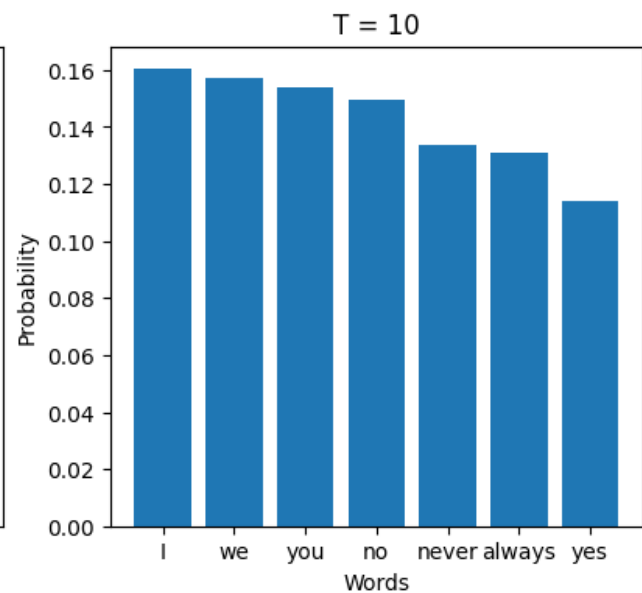
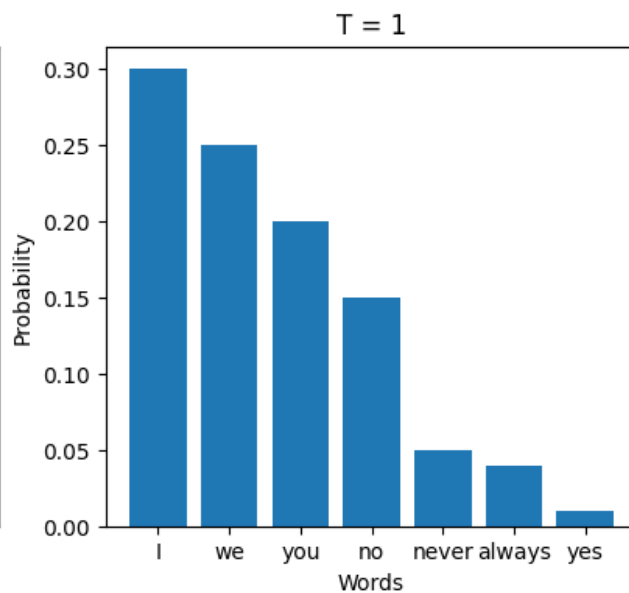
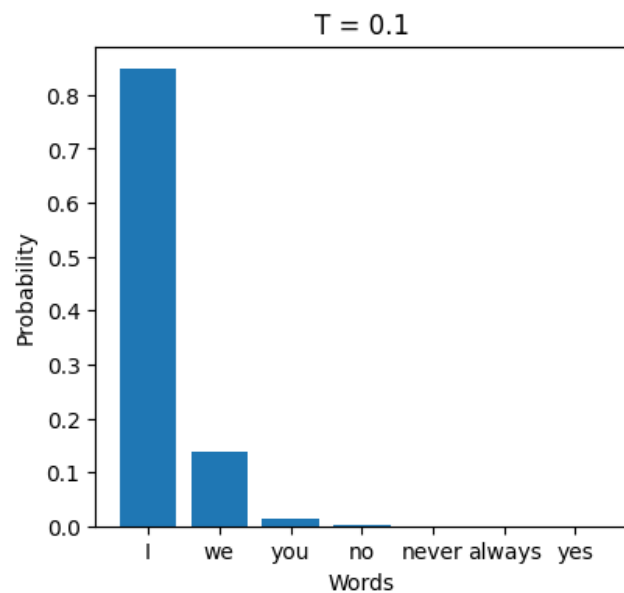

$$(x_1, \dots, x_V) \rightarrow (e^{x_1}, \dots, e^{x_V}) \rightarrow \left(\frac{e^{x_1}}{\sum_t e^{x_t}}, \dots, \frac{e^{x_V}}{\sum_t e^{x_t}} \right)$$

Make non-
negative

This is softmax

Temperature

$$\left(\frac{e^{x_1/T}}{\sum_t e^{x_t/T}}, \dots, \frac{e^{x_v/T}}{\sum_t e^{x_t/T}} \right)$$



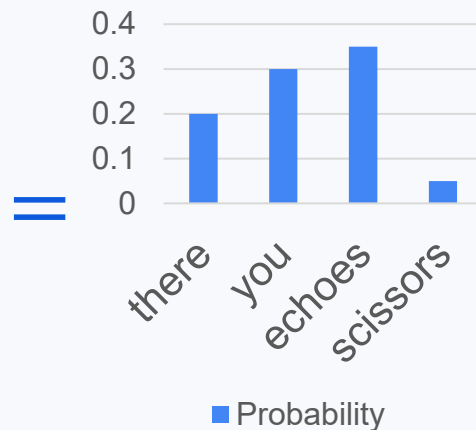
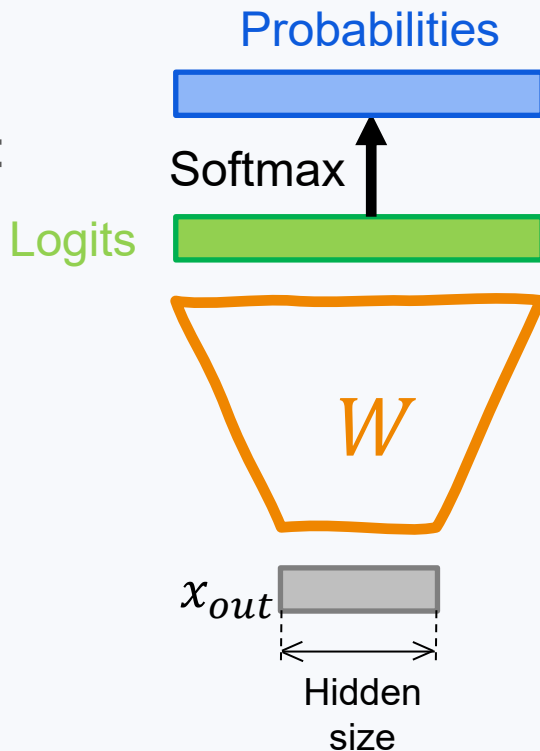
LM head

Basically, it is
a linear transformation:

$$\text{Logits} = x_{out} \cdot W$$

Probabilities

$$= \text{Softmax}(x_{out} \cdot W)$$



It's a Linear Classifier

It takes Features
and outputs

Class Probabilities

Training an LM: teacher forcing

Target Today I woke up tired <EOS>

Output

Hidden
states

h_0

Token
embeddings

Input <BOS>

What's bad with RNNs

- They are 100% **autoregressive**: process things token by token; not parallelizable. (Not that we solved this problem with transformers)
- They have to pack all the information from the sentence into a fixed-length vector. (Hence a plethora of memory management tactics)

Example: GRU (Gated Recurrent Unit)

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

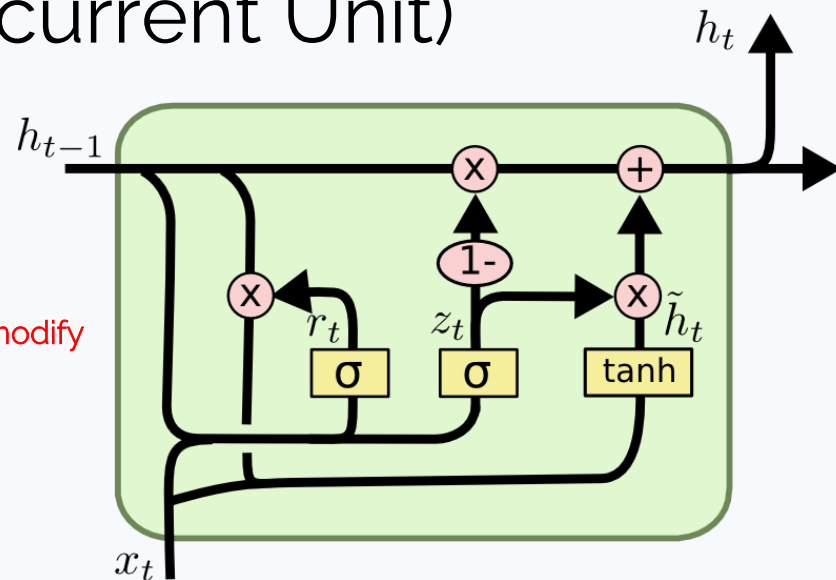
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$
 How much to modify

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

How much to forget (forget **gate**)

Modified information

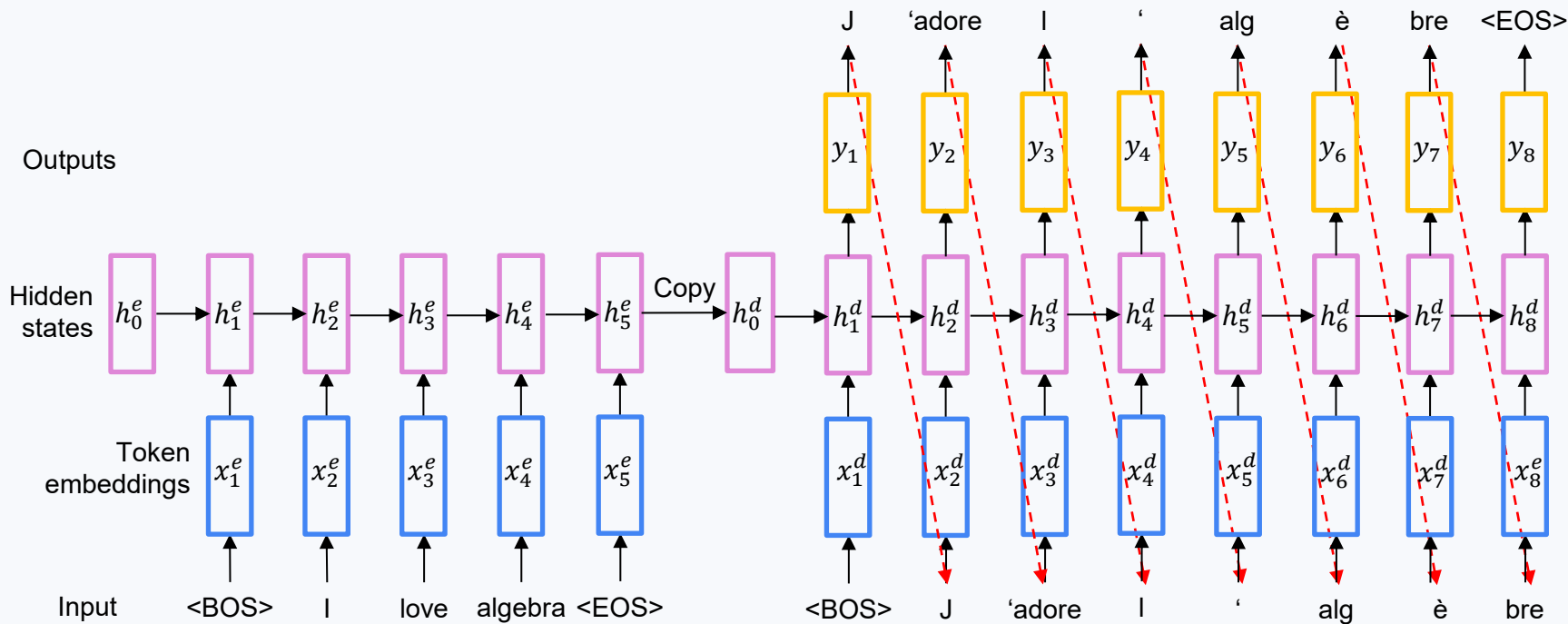


Seq2seq

Encoder-decoder

Encoder

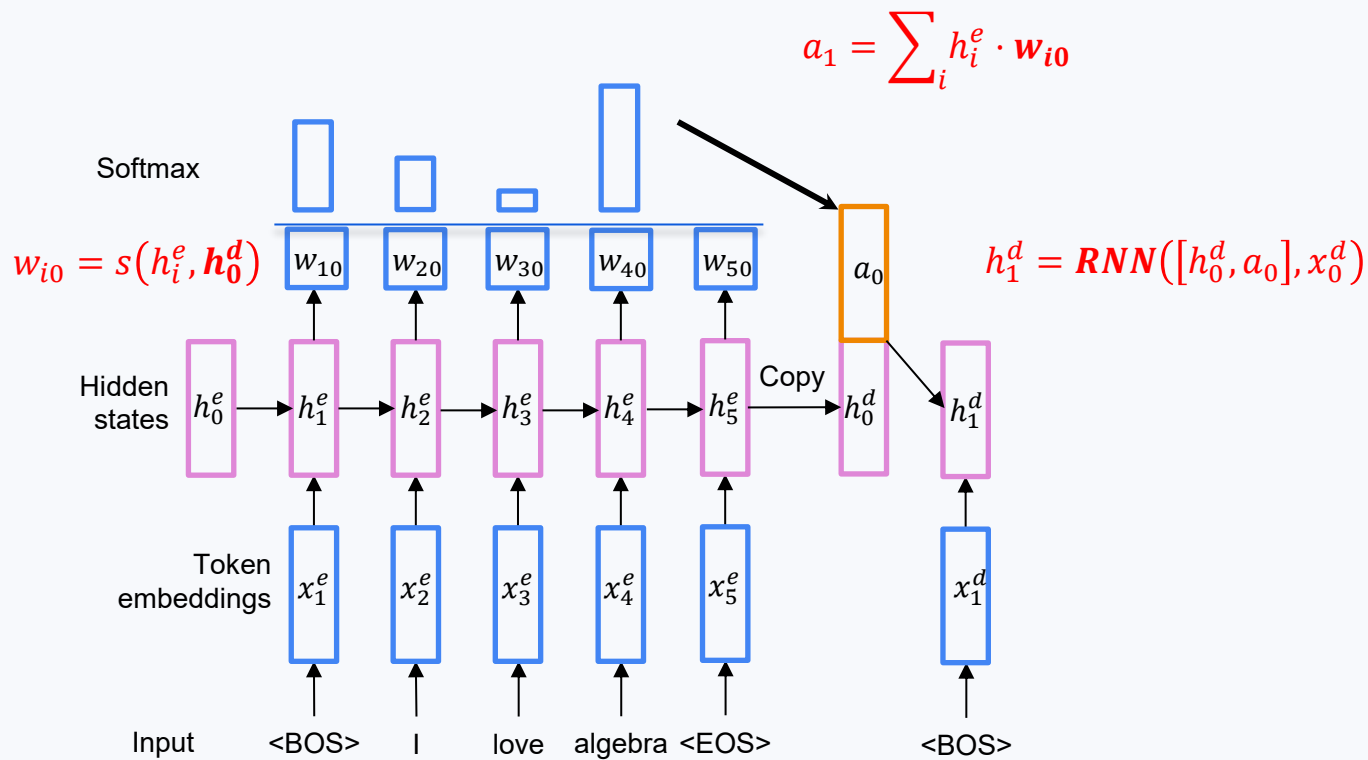
Decoder



(Bahdanau) attention

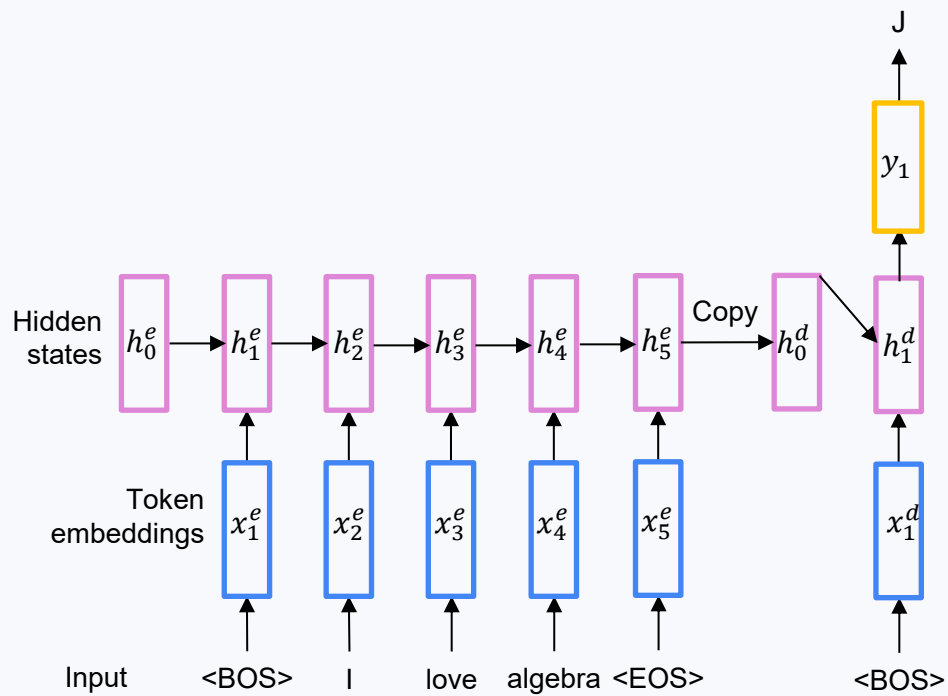
Encoder

Decoder



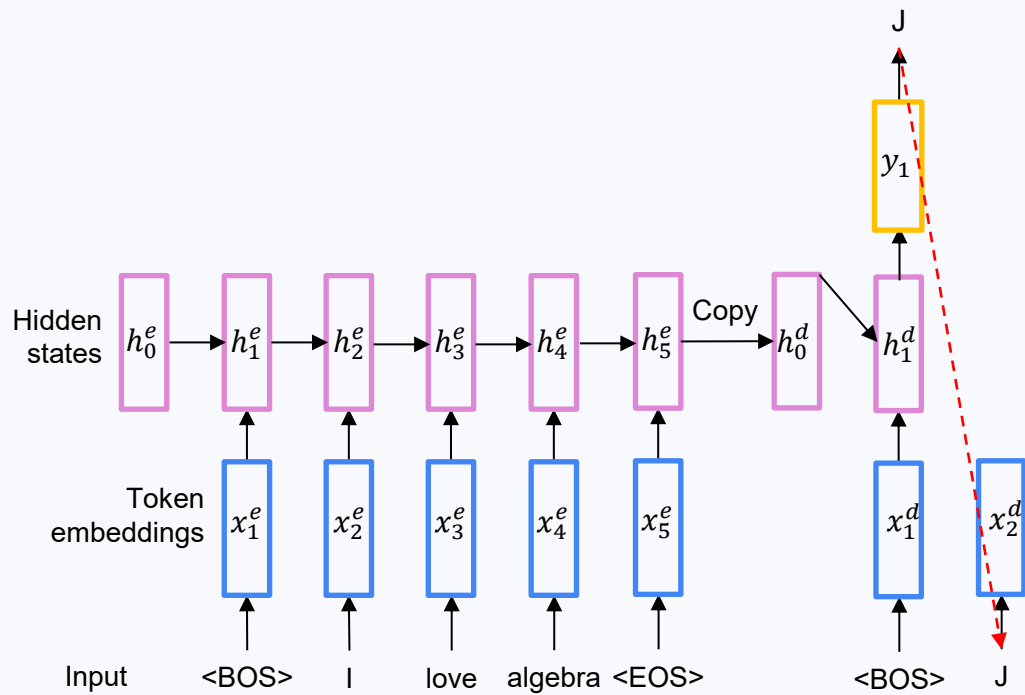
Encoder

Decoder



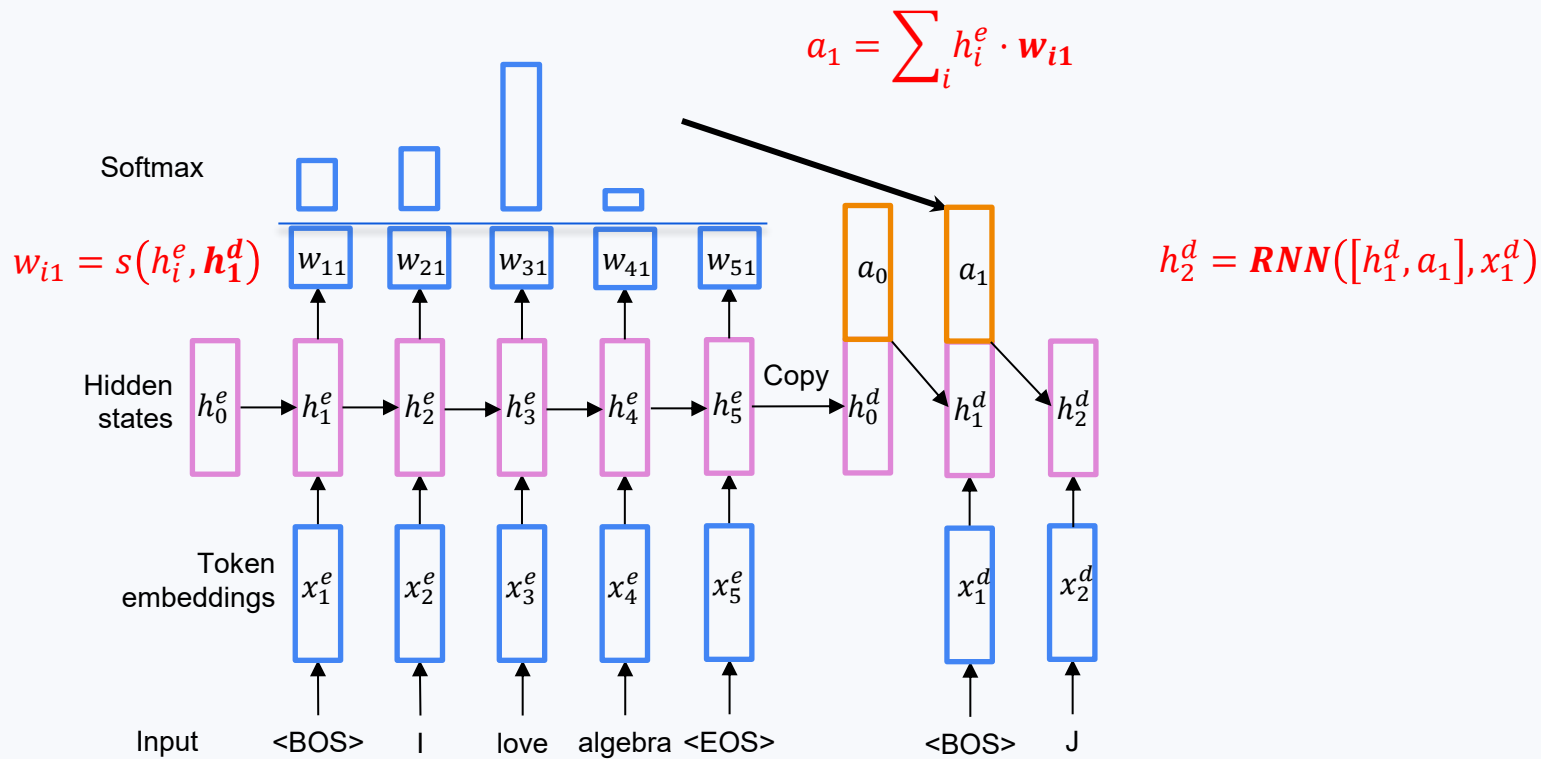
Encoder

Decoder



Encoder

Decoder



The effect of attention

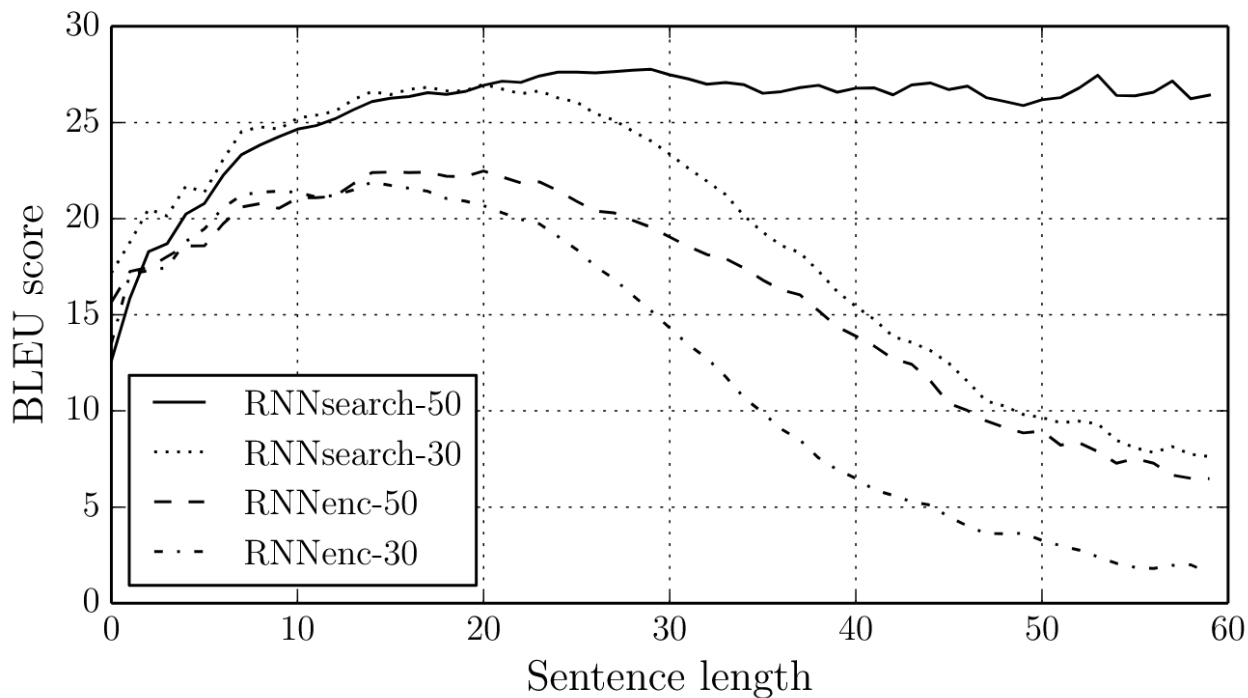
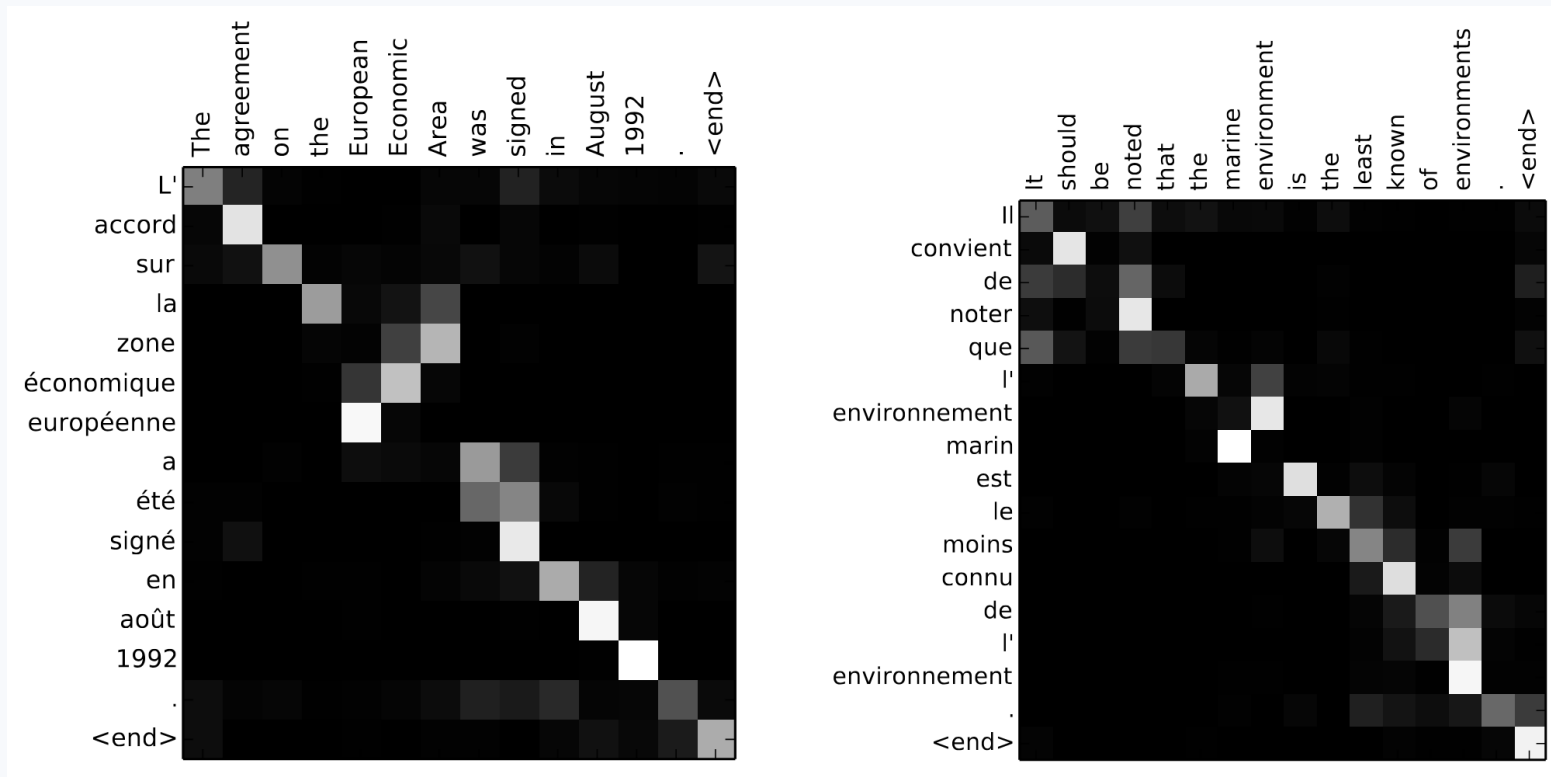


Figure 2: The BLEU scores of the generated translations on the test set with respect to the lengths of the sentences. The results are on the full test set which includes sentences having unknown words to the models.

Attention maps



Neural Machine Translation by Jointly Learning to Align and Translate

<https://arxiv.org/abs/1409.0473>